PBL Milestone 1: Algorithm Architecture Specification & Software Engineering Proposal

ALGOCOM (Algorithms and Complexity) / SOFTENG (Software Engineering) · Team Assignment

Metadata Field	Project Team Specification
Project Title	MetroFlow
Team Name	F4
Section & Year	III-CCSAD
Team Members & Roles	1. Lead Algorithm Architect: Lapating, Khent U. 2. Backend / Systems Engineer: De Vera, Josiah Ray S. 3. Data / QA Engineer: Vergara, John Gabriel D. 4. Systems Analyst / Technical Lead: Felipe, Marck Aaron V.
Repository URL	https://github.com/siahray/MetroFlow.git
Submission Date	November 4, 2026
Development Model	Waterfall (sequential phases with documented exit criteria)

Proposal at a glance

Computational problem. MetroFlow is a flood-response and rescue-dispatch system. Its core computation is a *vehicle-specific, flood-aware shortest-time path* on a live road graph: given a rescue vehicle's ground clearance and the current standing-water depth on every road segment, find the fastest route that never enters a segment the vehicle cannot cross.

Primary algorithm. A* search with a min-heap frontier, an admissible and consistent great-circle/top-speed heuristic, a hard clearance cut-off, and a depth-dependent speed-degradation edge cost [1], [2]. This is implemented in `backend/routing.py` (`td_a_star_search`).

Supporting algorithms. The backend contains 18 more algorithms beside A*: the hydrology model that generates the depths, passability rules, turn-by-turn steps, a dispatch preview, a nearest-node snap, the CR-004 flood-state resolver, report urgency ranking, a duplicate-neighbour map, a sensor/report mismatch check, evidence checks, a tamper-evident audit hash chain, a rate limiter, event replay, spatial queries, an advisory upsert, JWT sessions and the seed builder. Each is numbered ALG-02 to ALG-19 and gets the same treatment as A* inside every task: its own formal specification (Task 1), strategy table and pseudocode (Task 2), complexity and scalability table with a measured result (Task 3), interface contract (Task 4), story and work-package ticket with a benchmark assertion (Task 5), and answers (Task 6). A* is ALG-01 and the graded primary algorithm.

Waterfall lifecycle context

MetroFlow is built under a waterfall model: each phase finishes and passes its exit criteria before the next begins, and every later artifact traces back to an earlier one (BRD → FSD → TSD → code → tests). This milestone sits as follows:

Waterfall phase	Project artifact	Where it appears in this proposal

1. Requirements	BRD, FSD (e.g. FSD-FR-057, FSD-FR-089)	Task 1: problem, inputs, constraints, objective, output certificate
2. Design	TSD (ADR-004 A* routing; ADR-005 SCS-CN; §2 Algorithms)	Tasks 2–4: algorithm choice, pseudocode, complexity, architecture, API contract
3. Implementation	`backend/routing.py`, `backend/hydrology.py`	Task 5: work package DEV-ALGO-101 (engine is built; the benchmark module is the remaining item)
4. Verification	Test cases (TSD / FSD tables), PMP Testing Management exit criteria	Task 5 benchmark assertions; Task 6 edge-case answers
5. Acceptance	UAT scenarios, sign-off	Out of scope for Milestone 1

Contents. Task 1 Problem Modeling · Task 2 Paradigm & Pseudocode · Task 3 Complexity & Scalability · Task 4 Decoupled Architecture & API Schema · Task 5 Work Packages & Validation Plan · Task 6 Defense Questions · References · Presentation Deck (14 slides, appended).

> <!-- AI Assistance: We used Claude (Claude Code) to read our existing repository, draft the mathematical formulation, complexity derivations, pseudocode, JSON schemas, algorithm inventory and review answers in this document, and to lay out the report. The team reviewed each formula against `backend/routing.py`, `backend/hydrology.py`, `backend/reports.py` and `backend/audit.py`. -->

Task 1: Computational Problem Modeling & Formal Mathematical Formulation

Waterfall phase: Requirements. This task restates the routing requirements (FSD-FR-011 to 017, FSD-FR-089) as a mathematical problem.

1.1 Problem statement & domain context

During typhoon-driven flooding, standing water makes road segments impassable for some vehicles and merely slow for others. A rescue unit with 28 cm of ground clearance cannot use a lane that a 70 cm truck can. Routing that ignores depth and vehicle clearance sends units into submerged streets. The operational question is therefore: *for a given vehicle, what is the fastest route to a victim that only uses road segments the vehicle can actually cross, and does any such route exist?*

Flood depth is a standard predictor of road-network disruption: speed falls as depth rises and vehicles stop at a depth tied to their clearance [7]. Prior work on emergency services during floods assigns depths to road links and derives reduced vehicle speeds and service areas from them [6]. MetroFlow adopts the same modelling idea, using a per-vehicle clearance threshold and a depth-dependent speed penalty, and runs it on a live, sensor- and report-updated graph. Shortest-path search under constraints is a core graph problem [1], and goal-directed A* search is the standard way to cut the explored region on road networks [2], [5].

1.2 Flood-aware A* (ALG-01, primary): formal specification

Input parameters (I). A directed graph $G = (V, E)$ with $|V| = n$, $|E| = m$. Each node $u \in V$ has a coordinate $x_u = (\varphi_u, \lambda_u)$ (latitude, longitude). Each edge $e = (u, v) \in E$ has length $\ell_e > 0$ (m), posted speed $s_e > 0$ (km/h) and live water depth $d_e \geq 0$ (cm). A query supplies a source $s \in V$, target $t \in V$ and vehicle clearance $c > 0$ (cm).

Depths d_e are produced by the hydrology engine: for rainfall P (mm), curve number CN, $S = \frac{25400}{CN} - 254$ and initial abstraction $I_a = 0.2S$, the direct runoff is

$$Q = \begin{cases} 0 & P \leq I_a \\ \dfrac{(P - I_a)^2}{P - I_a + S} & P > I_a \end{cases}$$

which is then scaled to a street ponding depth in cm [8]. Verified hazards and sensor breaches can additionally close edges.

Boundary conditions & constraints (C).

- Passability: $d_e < c$ (an edge with $d_e \geq c$ is *submerged* and removed). The feasible subgraph is $G_c = (V, E_c)$, \(E_c=\{e: d_e
- $0 < c \leq 200$ cm (enforced at the API); $s, t \in V$.
- Effective speed floor: $v_e \geq 0.5$ m/s so every traversal time is finite.
- Depth and speed are in cm and km/h throughout (project unit policy).

Objective function (f). This is an **optimization problem** (minimization) over s–t paths in G_c. Define the depth-degraded speed and traversal time of an edge:

$$v_e = \max\!\left(\frac{s_e/3.6}{1+(d_e/10)^{1.8}},\ 0.5\right)\ [\mathrm{m/s}], \qquad w_e = \frac{\ell_e}{v_e}\ [\mathrm{s}]$$

$$\min_{P=(e_1,\ldots,e_k)\ \text{path}\ s\to t\ \text{in}\ G_c} T(P) = \sum_{i=1}^{k} w_{e_i}$$

A companion **decision problem** is: does an s–t path exist in G_c? This is the 404 / "no safe route" case.

Output certificate (O). A deterministic tuple $(\mathrm{status},\ P^*,\ T^*)$: `status` ∈ {OK, NO_SAFE_ROUTE, INVALID_NODE}; P^* the node sequence; $T^* = T(P^*)$ the minimal travel time. If no path exists, $P^* = \emptyset$ and $T^* = \infty$. Optimality is checkable: T^* must equal the Dijkstra reference value on the same G_c [4].

Dispatch extension. For an SOS with K candidate vehicles, each with clearance c_k and position s_k, the preview solves K independent instances and ranks units by T_k^*.

<!-- AI Assistance: Claude helped formalise the objective function and the SCS-CN statement from our code. The speed-degradation exponent 1.8 and the 0.5 m/s floor come from our existing implementation. They are modelling choices of the team, not values taken from [6] or [7]. -->

1.3 Formal specification of the supporting algorithms

MetroFlow contains more algorithms than the A* search. Each one below gets the same four-part specification (inputs, constraints, objective, output certificate) as A*. A* is ALG-01 and remains the graded primary algorithm; ALG-02 to ALG-19 support it or sit beside it in the same backend.

ALG-02: SCS-CN runoff and ponding depth (`backend/hydrology.py`)

Element	Specification
Input parameters (I)	Rainfall rate r (mm/h), duration D (h), curve number CN, node elevation z_u (m), ponding amplification α.
Boundary conditions & constraints (C)	$CN > 0$; $P = rD \geq 0$ (else runoff is 0); α from configuration; elevations above the 10 m datum are clamped; depth in cm, rainfall in mm.
Objective function (f)	Not an optimisation: a deterministic evaluation. $S = \frac{25400}{CN} - 254,\ I_a = 0.2S,\ Q = \frac{(P-I_a)^2}{P-I_a+S}$ for $P > I_a$, else 0; $d_u = \frac{Q}{10}\left(1 + 0.15(10 - \min(z_u, 10))\right)\alpha$ [8].
Output certificate (O)	Ponding depth $d_u \geq 0$ in cm, rounded to 0.01, for every node; written to the live edge states that A* reads.

ALG-03: Passability classification and hazard blocking (`backend/routing.py`)

Element	Specification
Input parameters (I)	Water depth d (cm), vehicle clearance c (cm), caution depth κ; a verified hazard's obstruction level.
Boundary conditions & constraints (C)	$c > 0$; levels in {CAUTION, LEVEL_2, LEVEL_3, IMPASSABLE}.
Objective function (f)	Classification: SUBMERGED iff $d \geq c$ (inclusive); LIGHT_FLOOD iff \(\kappa\le d
Output certificate (O)	A label in {PASSABLE, LIGHT_FLOOD, SUBMERGED} and a boolean $blocks$.

ALG-04: Turn-by-turn instruction generation (`backend/routing.py`)

Element	Specification
Input parameters (I)	A path $p_0, \ldots, p_k$, hop lengths and road names, node coordinates.
Boundary conditions & constraints (C)	Heading change $\leq 20^\circ$ is straight, $\leq 45^\circ$ slight, $\geq 135^\circ$ a U-turn; steps start with DEPART and end with ARRIVE.
Objective function (f)	Emit the shortest step list that hides no real turn. Bearing $\theta = \operatorname{atan2}(\sin\Delta\lambda\cos\varphi_2,\ \cos\varphi_1\sin\varphi_2 - \sin\varphi_1\cos\varphi_2\cos\Delta\lambda) \bmod 360^\circ$; heading change $\Delta = ((\theta_i - \theta_{i-1} + 540) \bmod 360) - 180$.
Output certificate (O)	Ordered steps (maneuver, road, distance, node ids, text).

ALG-05: Dispatch preview (one search per vehicle) (`backend/main.py`)

Element	Specification
Input parameters (I)	An SOS location, K candidate vehicles with position s_k and clearance c_k, the live graph.
Boundary conditions & constraints (C)	Only available units; a unit without a position cannot be previewed (409); no safe route is reported as 422.
Objective function (f)	For each vehicle solve A* from its nearest node to the SOS node; rank by T_k^* ascending.
Output certificate (O)	A list of (vehicle, T_k^*, route or reason) for the dispatcher.

ALG-06: Nearest-node snap (`backend/main.py (_nearest_node)`)

Element	Specification
Input parameters (I)	A coordinate x and the node set V with coordinates.
Boundary conditions & constraints (C)	$V \neq \emptyset$; great-circle distance in metres.
Objective function (f)	$\arg\min_{u \in V} \operatorname{hav}(x, x_u)$.
Output certificate (O)	The id of the closest road node.

ALG-07: Flood-state resolver (CR-004 priority rule) (`backend/flood_state.py`)

Element	Specification
Input parameters (I)	Per edge: the highest verified hazard (level, blocks?), live observations $(\text{source}, \text{kind}, t_{as_of}, \text{flagged})$, a NOAH class; plus now, a $raining$ flag, and a 30-minute freshness window.
Boundary conditions & constraints (C)	Fresh iff $-5\,\text{min} \leq now - t_{as_of} \leq 30\,\text{min}$; a live reading or NOAH never closes a road (BRULE-012, BRULE-016).
Objective function (f)	Choose the single applicable state by priority: (1) verified hazard; (2) best fresh flagged observation, ranked OBSERVED < MODELLED < SIMULATED then newest; (3) satellite (not built); (4) NOAH caution (penalty 1.1 / 1.3 / 1.6) only while raining; (5) NONE.

Output certificate (O)	`FloodState`: basis, source, kind, as_of, advisory, closes_road, route_penalty, notes.

ALG-08: Report urgency scoring and ranking (`backend/reports.py`)

Element	Specification
Input parameters (I)	k reports, each with depth level $\delta \in \{\text{ANKLE}, \text{KNEE}, \text{IMPASSABLE}\}$, source $\sigma \in \{\text{PHOTO}, \text{SENSOR}, \text{CALL}\}$, r nearby reports, age a (min).
Boundary conditions & constraints (C)	$\text{sev} = 1, 2, 3$; $\text{wt} = 1.0, 0.9, 0.8$; unknown source is weighted as CALL; $r < 0$ and $a < 0$ are treated as 0.
Objective function (f)	Order reports by descending $U(a) = U_0 e^{-a/30}$, $U_0 = \text{sev}(\delta)\,\text{wt}(\sigma)\,\min(1 + 0.25r, 2)$, so \(0
Output certificate (O)	The sorted moderation queue.

ALG-09: Duplicate-neighbour map for the moderation queue (`backend/main.py (_neighbour_map)`)

Element	Specification
Input parameters (I)	k reports with latitude, longitude, status and submit time.
Boundary conditions & constraints (C)	Neighbour = within 150 m and 15 minutes; only pending reports with a location participate.
Objective function (f)	For each report, the count of neighbours and the nearest neighbour.
Output certificate (O)	A map `report_id → (count, nearest id)`; the count feeds ALG-08 and the duplicate hint.

ALG-10: Sensor/report mismatch check and haversine distance (`backend/reports.py`)

Element	Specification
Input parameters (I)	A report location and gauges $(\text{lat}, \text{lon}, \text{has_reading}, \text{is_breach})$.
Boundary conditions & constraints (C)	Only gauges within 5,000 m count; $R = 6{,}371{,}000$ m.
Objective function (f)	Decision: does the nearest in-range gauge contradict the report? Haversine $a = \sin^2 \frac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \frac{\Delta\lambda}{2}$, $d = 2R\,\text{atan2}(\sqrt{a}, \sqrt{1-a})$.
Output certificate (O)	Boolean: true iff the nearest in-range gauge has a reading and is not in breach.

ALG-11: Evidence checks: digest, image type, EXIF, polygon, reporter hash (`backend/reports.py`)

Element	Specification
Input parameters (I)	An uploaded JPEG of b bytes, a claimed latitude/longitude, a user id and a salt.
Boundary conditions & constraints (C)	Pilot bounds lat 14.4–14.8, lon 120.9–121.2; hazard footprint radius 30 m.
Objective function (f)	Produce evidence metadata: SHA-256 of the bytes, sniffed image type, EXIF fields (make, capture time, GPS), a plausibility flag, a square hazard polygon, and a salted reporter hash.

Output certificate (O)	Digest, MIME, EXIF dict (or empty), plausibility boolean, WKT polygon, hex reporter hash.

ALG-12: Tamper-evident audit hash chain (`backend/audit.py`)

Element	Specification
Input parameters (I)	An ordered log $R_1, \ldots, R_L$ of attributable records.
Boundary conditions & constraints (C)	Append-only; each record serialised canonically (sorted keys, no whitespace).
Objective function (f)	Decision problem: is the chain intact? $h_0 = \perp,\ h_i = \text{SHA-256}(\text{canon}(R_i, h_{i-1}))$.
Output certificate (O)	`{valid, checked, broken_at}`.

ALG-13: Windowed rate limiter (`backend/rate_limit.py`)

Element	Specification
Input parameters (I)	A client id and route name (the bucket), the window W (s), and the limit λ.
Boundary conditions & constraints (C)	$\lambda \leq 0$ disables the check; the counter is atomic across serverless instances.
Objective function (f)	Count requests per bucket in the current window $w = \lfloor t/W \rfloor W$; reject when the count exceeds λ.
Output certificate (O)	The count and window start; HTTP 429 when over the limit.

ALG-14: SSE event replay and framing (`backend/events.py`)

Element	Specification
Input parameters (I)	A channel, a resume cursor $after$, a limit L_r; N rows in the event log.
Boundary conditions & constraints (C)	A malformed cursor is an error, not a silent reset; events are ordered by $event_id$.
Objective function (f)	Return events with $event_id > after$ in id order, up to L_r, then format each as an SSE frame.
Output certificate (O)	A list of events and `id/event/data` frames.

ALG-15: Sensor impact-radius edge marking (ST_DWithin) (`backend/main.py + PostGIS`)

Element	Specification
Input parameters (I)	A sensor reading $(\text{id},\ v,\ \text{unit},\ t)$; the sensor's location, threshold θ and impact radius ρ (m); the m road edges.
Boundary conditions & constraints (C)	Unit must match the sensor; timestamp not in the future; $v \geq 0$, finite; disabled sensors rejected.
Objective function (f)	$breach = (v \geq \theta)$; select $E_\rho = \{e :\ \text{dist}(e, \text{sensor}) \leq \rho\}$ and record the reading on each edge in E_ρ, flagged or not.
Output certificate (O)	The stored reading, the list of impacted edge ids, and an audit entry on breach.

ALG-16: Hazard coverage (ST_Intersects) (`backend/main.py + PostGIS`)

Element	Specification
Input parameters (I)	Active hazards H (polygon or a direct edge id) and the m road edges (line geometry).
Boundary conditions & constraints (C)	Only active hazards; a closure hazard may point at one edge without a polygon.
Objective function (f)	For each hazard, the set of edges its polygon intersects, unioned with its direct edge.
Output certificate (O)	A map $hazard_id \to$ set of edge ids; the router and the route-hazard report both read it.

ALG-17: Advisory upsert (last-writer-by-time) (`backend/main.py (_upsert_advisories)`)

Element	Specification
Input parameters (I)	r edge ids, a source, a kind, a flagged flag and the observation time as_of.
Boundary conditions & constraints (C)	One row per (edge, source); an older observation never overwrites a newer one.
Objective function (f)	Record the latest observation of the source on each edge.
Output certificate (O)	r upserted rows (no return value).

ALG-18: Session tokens (JWT HS256) (`backend/auth_utils.py`)

Element	Specification
Input parameters (I)	Claims (subject, role), a secret key, an expiry (default 24 h).
Boundary conditions & constraints (C)	HS256 only; the secret comes from the environment (a development fallback is reported at start-up).
Objective function (f)	Sign: $\mathrm{HMAC\text{-}SHA256}(\mathrm{header.payload},\ key)$. Verify: recompute and compare, check expiry.
Output certificate (O)	A compact token, or an authentication error.

ALG-19: Seed road-network builder (`backend/network_seed.py`)

Element	Specification
Input parameters (I)	The fixed spine nodes and edges, lattice rows and columns with names, coordinates and speeds.
Boundary conditions & constraints (C)	Edge endpoints must exist; ids unique; elevation derived per lattice cell.
Objective function (f)	Generate the pilot graph: lattice nodes plus bidirectional horizontal and vertical edges, then validate.
Output certificate (O)	A (nodes, edges) pair; the real seed has 34 nodes and 61 edges.

Task 2: Algorithmic Paradigm Selection & Logic Specification

Waterfall phase: Design (TSD §2 Algorithms, ADR-004). The choice below is already recorded as an accepted design decision. The template asks for one core algorithm, so A* is ALG-01; the other 18 algorithms in the system follow in 2.2 with the same schema.

Schema Field	Team Engineering Details
Algorithm Name	Clearance-Constrained, Depth-Penalized A* (implemented as `td_a_star_search`; "time-dependent" in the sense that edge costs are recomputed from the live depth state on every query, not that cost varies during one search).
Design Paradigm	Graph Algorithms & Network Theory: best-first search with an admissible, consistent heuristic over a min-heap priority queue.
Foundational References	Hart, Nilsson & Raphael (A*) [2]; Dechter & Pearl (optimality of A*) [3]; Dijkstra [4]; Cormen et al., Ch. 22 and 24 [1]; road-network context in Bast et al. [5].
Baseline Alternative	Exhaustive simple-path enumeration (DFS with no pruning), then take the minimum. Secondary baseline: Dijkstra ($h \equiv 0$), which is correct but explores in all directions.
Computational Justification	The number of simple s–t paths grows factorially in n, so enumeration is infeasible beyond toy graphs. A* returns the same optimum in $O(m \log n)$ and, with a consistent heuristic, settles each node at most once; the heuristic additionally steers the frontier toward t, expanding fewer nodes than Dijkstra [1], [2], [3]. Measured (Section 3.5): 33–43% fewer nodes settled and identical cost on 120 of 120 queries, but about 18–52% *more* wall time in pure Python because the haversine heuristic is costly.
Invariants & Recurrence Relations	Bellman recurrence on the feasible graph: $g(s) = 0, \quad g(v) = \min_{(u,v)\in E_c} \big(g(u) + w_{uv}\big)$. Priority: $f(v) = g(v) + h(v)$. Heuristic: $h(v) = \dfrac{\mathrm{hav}(x_v, x_t)}{v_{\max}}$, $v_{\max} = \max_e s_e / 3.6$. **Admissible:** $w_{uv} \geq \ell_{uv}/v_{\max} \geq \mathrm{hav}(x_u, x_v)/v_{\max}$ (the penalty factor is ≥ 1), so h never overestimates the remaining time. **Consistent:** $h(u) \leq w_{uv} + h(v)$ by the triangle inequality of great-circle distance, so f is non-decreasing along any path and the first time a node is popped its g is optimal [2], [3]. Loop invariant: every node in `settled` has final optimal g; every heap entry has $g \geq g_{\text{best}}$.

2.1 Flood-aware A* (ALG-01, primary): structured pseudocode

```
TD-ASTAR(G, coords, s, t, c):                  // G: adjacency u -> [(v, len_m, speed_kmh,
depth_cm)]
 1  if s not in coords or t not in coords: return (INVALID_NODE, [], inf)
 2  if s == t: return (OK, [s], 0)
 3  v_max  ← max(max over all edges of speed_kmh / 3.6, 0.1)          // m/s, fixed for this
query
 4  h(x)   ← haversine(coords[x], coords[t]) / v_max
 5  g[x]   ← inf for all x;  g[s] ← 0;  parent ← {};  settled ← {}
 6  PQ     ← min-heap { (h(s), 0, s) }                                  // (f, g, node)
 7  while PQ not empty:
 8      (f, gu, u) ← POP-MIN(PQ)
 9      if u in settled or gu > g[u]: continue                          // stale duplicate
10      settled ← settled ∪ {u}
11      if u == t: return (OK, RECONSTRUCT(parent, s, t), g[t])
12      for (v, len, speed, depth) in G[u]:
13          if depth ≥ c: continue                                        // hard clearance
cut-off
14          pen ← 1 + (depth / 10)^1.8                                    // depth penalty,
pen ≥ 1
15          v_eff ← max((speed / 3.6) / pen, 0.5)                         // m/s, floor keeps
cost finite
16          g' ← gu + len / v_eff
17          if g' < g[v]: g[v] ← g'; parent[v] ← u; PUSH(PQ, (g' + h(v), g', v))
18  return (NO_SAFE_ROUTE, [], inf)                                          // frontier
exhausted: t unreachable in G_c

RECONSTRUCT(parent, s, t): follow parent[] from t back to s; reverse.
```

Termination: each node is settled at most once and edges are finite, so the loop ends in at most $m + 1$ pops; line 18 is the termination certificate for the infeasible case.

> <!-- AI Assistance: Claude produced the pseudocode by reading `td_a_star_search` and wrote the admissibility/consistency argument. The team should be able to re-derive both on the board. Note: an earlier version of our code used a hard-coded 50 km/h in h and was inadmissible; our regression fuzz test in `tests/test_routing.py` guards that fix. -->

2.2 Strategy specification and pseudocode for the supporting algorithms

Each supporting algorithm completes the same Strategy Specification Schema as A*, followed by its pseudocode.

ALG-02: SCS-CN runoff and ponding depth (`backend/hydrology.py`)

Schema Field	Team Engineering Details
Algorithm Name	SCS-CN runoff and ponding depth (`backend/hydrology.py`)
Design Paradigm	Closed-form empirical hydrology (NRCS curve-number method) [8].
Foundational References	USDA NRCS National Engineering Handbook Part 630, ch. 10 [8].
Baseline Alternative	A 2-D hydrodynamic shallow-water simulation (rejected in TSD ADR-005): cost grows with grid cells times time steps and needs terrain and drainage data the pilot does not have.
Computational Justification	$O(1)$ per node with a handful of parameters; explainable to dispatchers; adequate as a decision aid.

Invariants & Recurrence Relations	$Q = 0$ for $P \leq I_a$; Q is non-decreasing in P; \(Q

```
DEPTH-AT-NODE(r, D, CN, z, alpha):
  P ← r * D;  if P ≤ 0: return 0
  S ← 25400 / CN - 254;  Ia ← 0.2 * S
  if Ia ≥ P: return 0
  Q ← (P - Ia)^2 / (P - Ia + S)
  f ← 1 + 0.15 * (10 - min(z, 10))
  return round(max(0, (Q / 10) * f * alpha), 2)
```

ALG-03: Passability classification and hazard blocking (`backend/routing.py`)

Schema Field	Team Engineering Details
Algorithm Name	Passability classification and hazard blocking (`backend/routing.py`)
Design Paradigm	Threshold classification (rule table).
Foundational References	Flood depth against vehicle clearance as the stopping criterion [7]; project rule BRULE-008 and TSD §2.6.
Baseline Alternative	The router and the map legend each keep their own threshold, which can drift apart.
Computational Justification	One function defines the labels, so legend colour and routing cannot disagree. $O(1)$.
Invariants & Recurrence Relations	**Caveat found while reading the code:** `td_a_star_search` applies the same inclusive test inline (`depth ≥ clearance`) instead of calling `classify_passability`, so the two agree by inspection, not by construction.

```
CLASSIFY(d, c, kappa):
  if d ≥ c: return SUBMERGED
  if d ≥ kappa: return LIGHT_FLOOD
  return PASSABLE
BLOCKS(level): return level in {LEVEL_2, LEVEL_3, IMPASSABLE}
```

ALG-04: Turn-by-turn instruction generation (`backend/routing.py`)

Schema Field	Team Engineering Details
Algorithm Name	Turn-by-turn instruction generation (`backend/routing.py`)
Design Paradigm	Single-pass geometric classification with thresholds; no search.
Foundational References	Great-circle bearing formulae; textbook trigonometry [1].
Baseline Alternative	One step per hop (k steps), announcing a turn at every block on a gently bending road.
Computational Justification	Merging bends gives fewer, usable steps in $O(k)$.
Invariants & Recurrence Relations	Steps partition the path: their distances sum to the path length, and an interior node starts a step only if $\lvert\Delta\rvert > 20^\circ$.

```
STEPS(path, hops, coords):
  bearings ← [bearing(coords[a], coords[b]) for (a, b, ...) in hops]
  bounds ← [(0, DEPART)]
  for i in 1 .. len(path)−2:
      m ← classify_turn(bearings[i−1], bearings[i])
      if m != STRAIGHT: bounds.append((i, m))
  bounds.append((len(path)−1, ARRIVE))
  return [step(bounds[j], bounds[j+1]) for j]       // distance = sum of hops between bounds
```

ALG-05: Dispatch preview (one search per vehicle) (`backend/main.py`)

Schema Field	Team Engineering Details
Algorithm Name	Dispatch preview (one search per vehicle) (`backend/main.py`)
Design Paradigm	Repeated single-source search (one A* run per candidate); aggregation by sorting.
Foundational References	A* [2], [3]; sorting [1].
Baseline Alternative	One multi-source Dijkstra from the SOS node (equivalent only if all vehicles shared one clearance).
Computational Justification	Clearances differ per vehicle, so each needs its own feasible subgraph G_{c_k}; K is small.
Invariants & Recurrence Relations	Each result is the optimum for its own clearance; the ranking is by travel time only.

```
PREVIEW(sos, vehicles, G):
  t ← nearest_node(sos);  out ← []
  for v in vehicles:
      if v.position is null: out.append((v, no−position)); continue
      s ← nearest_node(v.position)
      (path, T) ← TD−ASTAR(G, coords, s, t, v.clearance_cm)
      out.append((v, T, path) if T < inf else (v, no−safe−route))
  return sort(out by T)
```

ALG-06: Nearest-node snap (`backend/main.py (_nearest_node)`)

Schema Field	Team Engineering Details
Algorithm Name	Nearest-node snap (`backend/main.py (_nearest_node)`)
Design Paradigm	Linear scan with a minimum.
Foundational References	Nearest-neighbour search; k-d trees [1].
Baseline Alternative	A k-d tree or grid index: $O(\log n)$ per query after an $O(n \log n)$ build.
Computational Justification	No index to build or keep in sync with a changing graph; adequate for the pilot graph of 34 nodes.
Invariants & Recurrence Relations	The returned node minimises the haversine distance; ties keep the first in iteration order.

```
NEAREST(x, nodes):
  best ← null; bd ← inf
  for u in nodes:
      d ← haversine(x, nodes[u])
      if d < bd: bd ← d; best ← u
  return best
```

ALG-07: Flood-state resolver (CR-004 priority rule) (`backend/flood_state.py`)

Schema Field	Team Engineering Details
Algorithm Name	Flood-state resolver (CR-004 priority rule) (`backend/flood_state.py`)
Design Paradigm	Ordered rule list (first match wins) with a lexicographic minimum over fresh observations.
Foundational References	CR-004 §3.3 (project change request); sorting and selection [1].
Baseline Alternative	Average or vote across sources (lets a modelled value outvote a gauge) or newest-wins (lets a simulated sensor override a measurement).
Computational Justification	Predictable, explainable precedence; each state carries why it was chosen.
Invariants & Recurrence Relations	Only a verified blocking hazard closes a road. Missing data is never reported as safe: with nothing flagged the basis is NONE and `has_live_data` says whether anything covers the edge. A stale flagged reading is ignored but recorded.

```
RESOLVE(e, now, raining, window):
  if e.hazard_level: return state(VERIFIED_HAZARD, closes = e.hazard_blocks)
  fresh ← [o in e.live if o.flagged and is_fresh(o.as_of, now, window)]
  if fresh: b ← min(fresh, key = (kind_rank(o.kind), −o.as_of)); return state(LIVE_ADVISORY,
b)
  if e.noah_class:
      return state(NOAH_CAUTION, penalty) if raining else state(NOAH_LAYER)
  return state(NONE)
```

ALG-08: Report urgency scoring and ranking (`backend/reports.py`)

Schema Field	Team Engineering Details
Algorithm Name	Report urgency scoring and ranking (`backend/reports.py`)
Design Paradigm	Weighted scoring with exponential decay, then comparison sort.
Foundational References	Comparison sorting [1]; TSD §2.8.
Baseline Alternative	First-in-first-out, or severity alone: ignores evidence strength, corroboration and staleness.
Computational Justification	An old impassable report no longer outranks a fresh, well-corroborated one; corroboration saturates at 2× so a crowd cannot inflate a score.
Invariants & Recurrence Relations	U is non-decreasing in severity, source weight and r (to the cap) and strictly decreasing in age; the queue scores live, because order between reports of different age changes over time.

```
RANK(reports, now):
  for r in reports:
      U0 ← sev[r.level] * wt[r.source] * min(1 + 0.25 * max(r.neighbours, 0), 2.0)
      r.U ← U0 * exp(−max(age_minutes(r, now), 0) / 30)
  return sort(reports by U descending)
```

ALG-09: Duplicate-neighbour map for the moderation queue (`backend/main.py (_neighbour_map)`)

Schema Field	Team Engineering Details
Algorithm Name	Duplicate-neighbour map for the moderation queue (`backend/main.py (_neighbour_map)`)
Design Paradigm	All-pairs comparison with a distance and time filter.
Foundational References	Spatial bucketing / grid hashing [1].
Baseline Alternative	Bucket reports into 150 m grid cells and compare only adjacent cells: expected $O(k)$.
Computational Justification	Simple and correct; acceptable only because the queue page is capped at 200 rows.
Invariants & Recurrence Relations	Symmetry: haversine and the time window are symmetric, so a counts b iff b counts a ; a report never counts itself.

```
NEIGHBOURS(rows, now):
  pending ← [r in rows if r.status == PENDING and r.has_location]
  for r in rows:
      if r not in pending: result[r] ← (0, null); continue
      cnt ← 0; nearest ← null
      for o in pending, o != r:
          if |o.time − r.time| > 15 min: continue
          d ← haversine(r, o);  if d > 150: continue
          cnt ← cnt + 1;  update nearest by d
      result[r] ← (cnt, nearest)
```

ALG-10: Sensor/report mismatch check and haversine distance (`backend/reports.py`)

Schema Field	Team Engineering Details
Algorithm Name	Sensor/report mismatch check and haversine distance (`backend/reports.py`)
Design Paradigm	Linear nearest-neighbour scan plus a closed-form great-circle distance.
Foundational References	Great-circle distance is a metric (triangle inequality), the basis of the A* heuristic's consistency [1].
Baseline Alternative	Flag on *any* contradicting gauge (a distant gauge could override a close one); a k-d tree gives $O(\log g)$ after an $O(g \log g)$ build [1].
Computational Justification	g is small (tens of gauges), so the scan is cheaper than an index.
Invariants & Recurrence Relations	The selected gauge minimises distance (strict <, first wins on ties). Absence of data is not a contradiction: no coordinates, no gauge in range, or no reading all return false.

```
MISMATCH(lat, lon, gauges):
  if lat or lon is null: return false
  nearest ← null
  for (glat, glon, has, breach) in gauges:
      d ← haversine(lat, lon, glat, glon)
      if d ≤ 5000 and (nearest is null or d < nearest.d): nearest ← (d, has, breach)
  return nearest is not null and nearest.has and not nearest.breach
```

ALG-11: Evidence checks: digest, image type, EXIF, polygon, reporter hash (`backend/reports.py`)

Schema Field	Team Engineering Details
Algorithm Name	Evidence checks: digest, image type, EXIF, polygon, reporter hash (`backend/reports.py`)
Design Paradigm	Hashing plus a bounded binary-format parser (TIFF image-file directories).
Foundational References	SHA-256 [16]; JPEG/EXIF segment layout.
Baseline Alternative	A full image library: heavier dependency than the few tags needed.
Computational Justification	Reads only the APP1 segment and stops at the start-of-scan marker, so cost does not depend on image size.
Invariants & Recurrence Relations	A digest change means the bytes changed; the parser never raises: any error degrades to \"no EXIF\"; the reporter hash is salted so identity never leaves the function.

```
READ-EXIF(data):
  if not starts_with(data, FFD8): return {}
  pos ← 2
  while pos + 4 ≤ len(data):
      if data[pos] != 0xFF: return {}
      marker ← data[pos+1];  if marker == 0xDA: break      // start of scan
      len ← u16(data[pos+2:]);  if len < 2: return {}
      if marker == 0xE1 and segment starts with "Exif": return parse_tiff(segment)
      pos ← pos + 2 + len
  return {}           // any IndexError / ValueError also returns {}
```

ALG-12: Tamper-evident audit hash chain (`backend/audit.py`)

Schema Field	Team Engineering Details
Algorithm Name	Tamper-evident audit hash chain (`backend/audit.py`)
Design Paradigm	Hash chain over a collision-resistant hash.
Foundational References	SHA-256 [16].
Baseline Alternative	Per-record checksum with no link to neighbours (detects an edit but not a delete or reorder), or a trusted full copy ($O(L)$ extra storage).
Computational Justification	Editing R_j changes h_j and every later hash, so the first mismatch is at j.
Invariants & Recurrence Relations	If every stored h_i equals its recomputation, no record or its order has changed since writing (assuming collision resistance). **Limit:** a writer who can rewrite the table can

	recompute the whole chain, and truncating the newest entries is not detectable without an external anchor.

```
VERIFY-CHAIN(entries):                    // ascending log_id
  prev ← null
  for i, e in enumerate(entries):
      body ← parse_json(e.payload)       // failure: return (false, i, e.log_id)
      exp  ← SHA256(canon(e.user_id, e.action_type, body, e.created_at, prev))
      if exp != e.hash_signature: return (false, i, e.log_id)
      prev ← e.hash_signature
  return (true, len(entries), null)
```

ALG-13: Windowed rate limiter (`backend/rate_limit.py`)

Schema Field	Team Engineering Details
Algorithm Name	Windowed rate limiter (`backend/rate_limit.py`)
Design Paradigm	Fixed-window counter held in the database (single atomic upsert).
Foundational References	TSD LIM-006 (project limit); PostgreSQL `INSERT ... ON CONFLICT` semantics.
Baseline Alternative	Sliding-window log ($O(\text{requests})$ memory) or token bucket.
Computational Justification	No read-then-write race and no per-instance state, which matters on serverless hosting.
Invariants & Recurrence Relations	Limit: a fixed window permits up to about 2λ requests across a window boundary.

```
CONSUME(bucket, W, now):
  w ← floor(now / W) * W
  count ← SQL: INSERT (bucket, w, 1) ON CONFLICT (bucket, w)
                    DO UPDATE SET count = count + 1 RETURNING count
  return (count, w)       // caller rejects if count > limit
```

ALG-14: SSE event replay and framing (`backend/events.py`)

Schema Field	Team Engineering Details
Algorithm Name	SSE event replay and framing (`backend/events.py`)
Design Paradigm	Indexed range scan on an append-only log.
Foundational References	SSE wire format; PostgreSQL B-tree range scans.
Baseline Alternative	Redis pub/sub or Postgres LISTEN/NOTIFY (not available behind the hosted pooler, ADR-024).
Computational Justification	Reconnecting clients resume exactly where they stopped; a rolled-back change leaves no event.
Invariants & Recurrence Relations	Event ids are monotonic; a committed change always has its event in the same transaction.

```
REPLAY(channel, after, limit):
  return SELECT * FROM event_log
         WHERE channel = :channel AND event_id > :after
         ORDER BY event_id LIMIT :limit          // uses index (channel, event_id)
FRAME(id, type, payload): "id: id\nevent: type\ndata: " + json(payload) + "\n\n"
```

ALG-15: Sensor impact-radius edge marking (ST_DWithin) (`backend/main.py + PostGIS`)

Schema Field	Team Engineering Details
Algorithm Name	Sensor impact-radius edge marking (ST_DWithin) (`backend/main.py + PostGIS`)
Design Paradigm	Spatial range query delegated to the database (PostGIS).
Foundational References	PostGIS `ST_DWithin` [14].
Baseline Alternative	Scan all m edges and compute haversine in application code: $O(m)$.
Computational Justification	With a usable spatial index the query is $O(\log m + r)$; the database also handles geography (metre) distances.
Invariants & Recurrence Relations	**Finding:** the query casts the geometry column to `geography`, which the existing geometry GiST index cannot serve, so it is a sequential scan: $O(m)$, not $O(\log m + r)$.

```
READING(sensor, v, t):
  breach ← v ≥ sensor.threshold
  E ← SELECT edge_id FROM road_edges
       WHERE ST_DWithin(geom::geography, sensor.geom::geography, sensor.impact_radius_m)
  upsert advisory (edge, SENSOR:id, kind, breach, t) for e in E        // ALG-17
  if breach: audit.record(SENSOR_BREACH, ..., E)
```

ALG-16: Hazard coverage (ST_Intersects) (`backend/main.py + PostGIS`)

Schema Field	Team Engineering Details
Algorithm Name	Hazard coverage (ST_Intersects) (`backend/main.py + PostGIS`)
Design Paradigm	Spatial join using the geometry index.
Foundational References	PostGIS `ST_Intersects` [14]; TSD §2.6.
Baseline Alternative	Test every hazard against every edge: $O(hm)$.
Computational Justification	A GiST index on the edge geometry makes each hazard $O(\log m + r)$; one mapping keeps router and report consistent about what blocks.
Invariants & Recurrence Relations	Router and route report use the same mapping, so \"blocked\" has one definition.

```
COVERAGE():
  H ← SELECT * FROM hazard_zones WHERE active
  for (h, e) in SELECT h.id, e.id FROM hazard_zones h JOIN road_edges e
                ON ST_Intersects(e.geom, h.geometry) WHERE h.active:
      covered[h].add(e)
  for h in H where h.edge_id: covered[h].add(h.edge_id)
  return H, covered
```

ALG-17: Advisory upsert (last-writer-by-time) (`backend/main.py (_upsert_advisories)`)

Schema Field	Team Engineering Details
Algorithm Name	Advisory upsert (last-writer-by-time) (`backend/main.py (_upsert_advisories)`)
Design Paradigm	Bulk `INSERT ... ON CONFLICT DO UPDATE ... WHERE as_of ≤ new.as_of`.
Foundational References	PostgreSQL upsert semantics; CR-004.
Baseline Alternative	Read each row, compare in application code, then write: $2r$ round trips and a race between instances.
Computational Justification	One statement, atomic, $O(r)$ regardless of the table size.
Invariants & Recurrence Relations	`as_of` is monotone per (edge, source): the stored value never moves backwards in time.

```
UPSERT(edge_ids, source, kind, flagged, as_of):
  if edge_ids is empty: return
  INSERT INTO edge_advisories (edge_id, source, kind, flagged, as_of) VALUES ...
  ON CONFLICT (edge_id, source) DO UPDATE SET kind, flagged, as_of, updated_at
  WHERE edge_advisories.as_of ≤ excluded.as_of
```

ALG-18: Session tokens (JWT HS256) (`backend/auth_utils.py`)

Schema Field	Team Engineering Details
Algorithm Name	Session tokens (JWT HS256) (`backend/auth_utils.py`)
Design Paradigm	Symmetric message authentication through a library.
Foundational References	JSON Web Token, RFC 7519 [17]; SHA-256 [16].
Baseline Alternative	Server-side sessions (state in the database on every request).
Computational Justification	Stateless verification, suitable for serverless hosting (ADR-021).
Invariants & Recurrence Relations	A token verifies only if it was signed with the key and has not expired. Not our own cryptography.

```
SIGN(claims, key, ttl):  payload ← claims + {exp: now + ttl};  return HS256(payload, key)
VERIFY(token, key):     decode with algorithms=[HS256]; on any error raise 401
```

ALG-19: Seed road-network builder (`backend/network_seed.py`)

Schema Field	Team Engineering Details
Algorithm Name	Seed road-network builder (`backend/network_seed.py`)
Design Paradigm	Deterministic generation of a grid graph.
Foundational References	Graph representation [1].
Baseline Alternative	Importing a real road extract (needed for city scale, out of scope for the pilot).

Computational Justification	Reproducible demo data for tests and the pilot.
Invariants & Recurrence Relations	After `validate`: every edge references existing nodes; ids are unique.

```
BUILD():
  nodes ← spine nodes;  edges ← spine edges
  for each (row, col): nodes.append(lattice_node(row, col))
  for each row, col < last: edges += both directions between (row,col) and (row,col+1)
  for each col, row < last: edges += both directions between (row,col) and (row+1,col)
  return nodes, edges
```

Task 3: Theoretical Asymptotic Analysis & Scalability Profiling

Waterfall phase: Design. The analysis is done before verification, so that the benchmarks in 3.5 and 3.6 have a prediction to be checked against.

3.1 Worst-case time $O(\cdot)$

1. **Initialisation** (lines 3–6): scanning all edges for $v_{\max}$ is $O(m)$; setting $g[x] = \infty$ for all nodes is $O(n)$. Subtotal $O(n + m)$.
2. **Settlement bound.** Because h is consistent (Task 2), a node's first valid pop carries its optimal g and it is never re-expanded [3]. Hence lines 10–17 run for each node at most once.
3. **Relaxations.** Line 12 iterates the out-edges of the settled node, so across the whole run each edge is examined at most once: at most m executions of lines 13–17, each $O(1)$ apart from the push.
4. **Heap operations.** Each successful relaxation pushes one entry, so at most $m + 1$ entries are ever pushed and popped. Each push or pop costs $O(\log |PQ|) \le O(\log(m + 1))$. Since $m \le n^2$, $\log m \le 2 \log n$, i.e. $O(\log n)$ [1].
5. **Total.** $T_{\text{worst}}(n, m) = O(n + m) + O\big((m + 1) \log n\big) = O(m \log n)$ for a connected road graph ($m \ge n - 1$). With $h \equiv 0$ the same bound is Dijkstra's with a binary heap [1], [4]; A* never does worse.

The master theorem does not apply: the algorithm is iterative, with no divide-and-conquer recurrence [1].

Baseline: exhaustive simple-path enumeration visits up to $(n - 2)!$ paths on a dense graph, i.e. $O(n!)$ [1].

3.2 Best- and average-case time $\Omega(\cdot)$, $E[\cdot]$

Best case. If h is near-perfect (a near-straight, uniform-speed corridor), the search only settles nodes on the path of length k: $\Omega(k \log n)$ plus the $O(n + m)$ initialisation done by the current code.

Average case. City road graphs are sparse ($m = \Theta(n)$, degree about 3–4) and near-planar. A goal-directed search settles only a region around the straight line $s \to t$, fewer than n nodes. Measured on a synthetic grid (Section 3.5): 33–43% fewer nodes than Dijkstra, and median runtime growing 3.5× and 4.3× for each 4× increase in nodes. This is a modest saving, not the corridor-only behaviour sometimes quoted for A* [5], because random speeds and flooding weaken the heuristic. We do not claim a closed-form bound. Heavy flooding lowers v_e while $v_{\max}$ is unchanged, which weakens h and moves the average case toward Dijkstra's.

3.3 Auxiliary space $S(\cdot)$

`g`, `parent`, `settled`: $O(n)$ each. Heap: lazy deletion allows stale duplicates, so up to $m + 1$ entries. No recursion (iterative loop, path rebuilt iteratively). Total $S = O(n + m)$. The graph itself is $O(n + m)$ input, not auxiliary.

3.4 Big Data (3 V's) evaluation

Dimension	Engineering Parameter	System Scalability Impact & Mitigation Strategy
Volume	Pilot: the seeded graph in `backend/network_seed.py`, which has only 34 nodes and 61 edges. Projected: a metro-scale graph, order 10^5 nodes and 4×10^5 directed edges (a planning figure, to be replaced by the real extract).	Python dict adjacency at that size is tens to low hundreds of MB. Mitigation: cache the static topology in the worker and refresh only the live depth overlay; load by bounding box (PostGIS `ST_DWithin` [14]) around s and t instead of the whole city; use compact arrays (CSR) if memory binds.
Velocity	Burst arrival during a typhoon: an SOS triggers K vehicle previews (one search each). Target: a few to tens of queries/s, with depth updates arriving from sensors and reports.	Searches are CPU-bound and independent. Run them in a worker pool (async queue) so request threads stay free; return a job id when a batch would exceed the request time budget. Searches read an immutable snapshot of the depth state, so concurrent writers cannot change costs mid-query.
Variety	Heterogeneous inputs: GeoJSON/PostGIS geometry, sensor readings, citizen reports, hazard polygons, rainfall values.	Normalise at the gateway into the one in-engine form (`depth_cm`, `speed_kmh`, `length_m`), validate with Pydantic, and keep unit conversion in a single place. The engine sees only numbers and node ids.

3.5 Measured results for A* (empirical check of 3.1–3.3)

The benchmark harness is `backend/benchmarks/bench_algorithms.py` (run `python -m benchmarks.bench_algorithms` from `backend/`). Timings are wall-clock medians and 95th percentiles of 3–7 runs on the development machine (macOS, Python 3.15.0, one process, fixed random seed). Inputs are synthetic: a square road grid with random speeds (30–60 km/h) and depths (0–40 cm). They are developer-machine numbers from one run, not CI numbers. The database-bound cases ran against a *throwaway* local PostGIS 15 / 3.3 container (linux/amd64 emulated on an ARM laptop, JIT off), created for the run and deleted afterwards; the hosted Supabase database was never touched, and the timings exclude network latency. The CI assertion given for each algorithm is roughly 5× the measured value and is not yet enforced in CI.

Nodes	Edges	A* median / p95 (ms)	Dijkstra median / p95 (ms)	Mean nodes settled: A* vs Dijkstra	Cost mismatches vs Dijkstra
1,024	3,968	0.97 / 2.26	0.64 / 1.21	358 vs 530	0 of 40
5,041	19,880	3.42 / 10.36	2.68 / 6.62	1,339 vs 2,184	0 of 40
19,881	78,960	14.56 / 45.67	12.33 / 28.35	5,461 vs 9,540	0 of 40

What the data supports. (1) *Optimality:* A* returned the same cost as the Dijkstra oracle on all 120 queries. (2) *Fewer expansions:* A* settled 33%, 39% and 43% fewer nodes than Dijkstra at the three sizes

and never more on any query. (3) *Growth:* A* median time grew 3.5× and 4.3× for 4× more nodes, close to linear on this sparse grid and well below the $O(m \log n)$ worst case. (4) The earlier 250 ms budget at N = 5,000 is met easily (p95 10.4 ms), so the CI assertion is tightened to p95 < 50 ms.

What the data does not support. A* was 18–52% *slower* in wall time than Dijkstra (52% at 1,024 nodes, 18% at 19,881) even though it settled fewer nodes. In pure Python the haversine call made for every heuristic evaluation costs more than the saved expansions, and the heuristic is weak when flooding slows edges well below the top speed. The accurate claim is *fewer expansions, same optimal cost*. A cheaper admissible bound (for example a precomputed equirectangular lower bound) or a compiled implementation would be needed to turn the saving into speed; this has not been tried.

3.6 Complexity and scalability of the supporting algorithms

Each supporting algorithm gets the same derivation rows as A* (worst case, best/average case, auxiliary space) plus the three Big Data dimensions, a measured result and its CI assertion.

ALG-02: SCS-CN runoff and ponding depth (`backend/hydrology.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	A fixed number of arithmetic operations: $O(1)$ per node, $O(n)$ for all n nodes. No loop or recursion inside the formula.
Average / best case	Same as worst case (data-independent).
Auxiliary space $S(\cdot)$	$O(1)$ extra; $O(n)$ to hold the resulting depths.
Volume	Order 10^5 nodes: one pass, 60 ms.
Velocity	Recomputed when a simulation is triggered or a rainfall input changes; independent per node, so it parallelises by node chunks.
Variety	Rainfall arrives as a request value or (CR-004) gauge readings; normalise to mm/h before the call.
Measured	6.0 ms for 10,000 nodes; 60.1 ms for 100,000 (10.0× for 10×: linear).
CI assertion	100,000 nodes in < 300 ms; $Q(P)$ monotone over a sweep of P.

ALG-03: Passability classification and hazard blocking (`backend/routing.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	$O(1)$ time and space per edge or hazard.
Average / best case	Same.
Auxiliary space $S(\cdot)$	$O(1)$.
Volume	Applied to every edge of the graph ($O(m)$ in total) when a state is rendered.
Velocity	Stateless, so any number of requests in parallel.
Variety	Input is a number and a string enum; the enum is validated at the gateway.
Measured	0.23 µs per classification plus block check (227 ms for 10^6 pairs).
CI assertion	For every d in a sweep, `classify(d, c) == SUBMERGED` iff A* excludes that edge (closes the caveat).

ALG-04: Turn-by-turn instruction generation (`backend/routing.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	k bearings, one boundary scan, and distance sums that add each hop once: $O(k)$ time, $O(k)$ space.
Average / best case	Same (data-independent).
Auxiliary space $S(\cdot)$	$O(k)$ for bearings and steps.
Volume	Paths are short (tens to thousands of nodes); 10,000 nodes take 6 ms.
Velocity	Computed per route response; stateless.
Variety	Input is the A* path plus road names; missing coordinates fall back to DEPART + ARRIVE.
Measured	0.064 / 0.545 / 6.11 ms at 100 / 1,000 / 10,000 nodes (8.5×, 11.2×: linear).
CI assertion	10,000-node path in < 30 ms; step distances sum to path length.

ALG-05: Dispatch preview (one search per vehicle) (`backend/main.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	K searches plus K node snaps plus a sort: $O\big(K(m \log n + n) + K \log K\big)$ (the snap is $O(n)$, ALG-06).
Average / best case	$K\times$ the A* average.
Auxiliary space $S(\cdot)$	$O(n + m)$ per search, reused across vehicles; $O(K)$ results.
Volume	K is the fleet size (tens); graph order 10^5 nodes.
Velocity	Burst when an SOS arrives; the K searches are independent, so they run in parallel workers.
Variety	Vehicle positions (telemetry), clearances, SOS coordinates: normalised to node ids before the engine.
Measured	38.8 ms for $K = 10$ at 5,041 nodes (3.9 ms per vehicle, sequential).
CI assertion	$K = 10$, $N = 5{,}041$ < 200 ms.

ALG-06: Nearest-node snap (`backend/main.py (_nearest_node)`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	n haversine evaluations: $O(n)$ time, $O(1)$ space.
Average / best case	Same (no early exit).
Auxiliary space $S(\cdot)$	$O(1)$.
Volume	At 10^5 nodes a snap costs 50 ms, the slowest step of a dispatch preview; a grid or k-d tree would remove it.
Velocity	Called once per unit and per SOS; stateless.
Variety	Input is a latitude/longitude pair; validated at the gateway.

Measured	5.05 ms per call at 10,000 nodes; 50.3 ms at 100,000 (linear). Raises `ValueError` on an empty node set.
CI assertion	10,000 nodes < 25 ms.

ALG-07: Flood-state resolver (CR-004 priority rule) (`backend/flood_state.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	Per edge: one filter and one minimum over o observations, $O(o)$; for m edges $O(m\bar{o})$, with $\bar{o}$ small.
Average / best case	$\bar{o} \leq 3$ in practice, so near-linear in m.
Auxiliary space $S(\cdot)$	$O(1)$ extra per edge; pure function with no database or clock access.
Volume	Order 10^5 edges: 280 ms per full pass.
Velocity	Computed on request for the edges in view; edges are independent, so shard by edge id.
Variety	Heterogeneous sources (gauge, modelled river value, simulated sensor, NOAH class) are normalised into `Observation` before the call.
Measured	20.4 ms for 10,000 edges; 279.6 ms for 100,000 (13.7× for 10×).
CI assertion	100,000 edges < 1.5 s; no result has `closes_road` true unless a blocking verified hazard was supplied.

ALG-08: Report urgency scoring and ranking (`backend/reports.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	Scoring is $O(1)$ per report; a comparison sort needs $\Omega(k \log k)$ and Timsort achieves $O(k \log k)$ [1]. Total $O(k \log k)$.
Average / best case	Same; near-linear when the list is already partly ordered.
Auxiliary space $S(\cdot)$	$O(k)$.
Volume	The page cap is 200 rows; 100,000 reports score and sort in 101 ms.
Velocity	Re-scored on every queue read because the decay depends on the clock.
Variety	Reports arrive from photos, gauges and calls; weights normalise the source.
Measured	9.8 ms for 10,000 reports; 101.1 ms for 100,000 (10.3× for 10×).
CI assertion	100,000 reports < 500 ms; stable order for equal scores.

ALG-09: Duplicate-neighbour map for the moderation queue (`backend/main.py (_neighbour_map)`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	Nested loop over pending reports: $\Theta(k^2)$ haversine checks.
Average / best case	Same order; the time filter skips some distance computations but not the pair visits.
Auxiliary space $S(\cdot)$	$O(k)$.

Volume	Cost: 34 ms at the 200-row page cap, 8.7 s at 3,200 reports, so a city-wide backlog would need the grid.
Velocity	Recomputed on each queue read; cache per queue version if reads are frequent.
Variety	Locations may be missing (call-in reports): those rows are skipped.
Measured	33.8 / 536 / 8,733 ms at 200 / 800 / 3,200 reports (15.8×, 16.3× per 4×: quadratic, as derived).
CI assertion	200 reports < 150 ms; symmetric counts.

ALG-10: Sensor/report mismatch check and haversine distance (`backend/reports.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	g haversines, each $O(1)$: $O(g)$ time per report, $O(1)$ space.
Average / best case	Same (no early exit).
Auxiliary space $S(\cdot)$	$O(1)$.
Volume	1,000 gauges cost 0.41 ms per report.
Velocity	Per report read; stateless.
Variety	Gauge readings and report coordinates normalised to decimal degrees.
Measured	Haversine 0.42 µs per call; mismatch check 4.2 / 41.7 / 411 µs at 10 / 100 / 1,000 gauges (linear).
CI assertion	1,000 gauges < 2 ms per call; haversine of a point with itself is 0.

ALG-11: Evidence checks: digest, image type, EXIF, polygon, reporter hash (`backend/reports.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	Digest $O(b)$; MIME sniff $O(1)$; EXIF $O(e)$ in the number of directory entries (bounded by b); polygon, plausibility and reporter hash $O(1)$.
Average / best case	EXIF stops at the first image-data marker, so it does not grow with image size.
Auxiliary space $S(\cdot)$	$O(b)$ for the held upload; parser $O(1)$ extra.
Volume	Uploads are megabytes; 5 MB digests in 2.2 ms.
Velocity	Per upload; CPU-light, so safe inside the request but bounded by an upload size limit.
Variety	Camera JPEGs of differing EXIF layouts (big/little endian, optional tags).
Measured	SHA-256 0.45 ms / 1 MB, 2.19 ms / 5 MB (linear). Real EXIF JPEG parsed in about 12 µs at 0, 1 and 5 MB bodies. Truncated EXIF returned `{}`. Polygon + plausibility 2.9 µs; reporter hash 0.47 µs.
CI assertion	5 MB digest < 50 ms; EXIF parse < 1 ms; truncated EXIF returns `{}`.

ALG-12: Tamper-evident audit hash chain (`backend/audit.py`)

Measure	Derivation and measurement

Worst-case time $O(\cdot)$	Append: one hash of $\lvert R_i \rvert$ bytes plus a lookup of the newest hash, $O(\lvert R_i \rvert)$. Verify: L hashes, $O(L\lvert\bar{R}\rvert)$ time.
Average / best case	Same (data-independent).
Auxiliary space $S(\cdot)$	$O(1)$ extra if streamed; the current code takes a list, $O(L)$.
Volume	100,000 entries verify in 467 ms.
Velocity	Appends are serialised by the chain (each needs the previous hash), so write rate is bounded by one row at a time; verification is on demand.
Variety	Payloads are arbitrary JSON; canonical serialisation makes the hash independent of key order.
Measured	4.9 / 46.2 / 467.5 ms at 1k / 10k / 100k entries (9.5×, 10.1×: linear); a one-entry edit was reported at exactly that entry.
CI assertion	100,000 entries < 2.5 s; tampering reported at the modified `log_id`.

ALG-13: Windowed rate limiter (`backend/rate_limit.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	$O(1)$ computation plus one database round trip.
Average / best case	Same.
Auxiliary space $S(\cdot)$	One row per (bucket, window); old rows pruned at start-up.
Volume	Rows grow with distinct clients times windows; pruned.
Velocity	One write per protected request, so it adds database load proportional to traffic.
Variety	Client identity comes from forwarding headers (platform-overwritten on Vercel).
Measured	0.96 ms median, 1.39 ms p95 per call on a local database (hosted adds network latency).
CI assertion	p95 < 10 ms locally.

ALG-14: SSE event replay and framing (`backend/events.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	$O(\log N + r)$ for r returned rows with the (channel, event_id) index; framing $O(\lvert\text{payload}\rvert)$ per event.
Average / best case	Same.
Auxiliary space $S(\cdot)$	$O(r)$ per replay.
Volume	Log grows without bound unless pruned; cost stays flat (see below).
Velocity	One relay per connected client tails the table; load scales with clients.
Variety	Payloads are JSON for hazards, vehicles and SOS requests.
Measured	1.1–1.2 ms for 100 events at 10k, 100k and 500k log rows, for both a tail read and a read from the start (flat in N). Framing 2.2 µs per event.

CI assertion	p95 < 20 ms at 500,000 rows.

ALG-15: Sensor impact-radius edge marking (ST_DWithin) (`backend/main.py + PostGIS`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	As deployed (sequential scan): $O(m)$ distance checks per reading. With a matching spatial index: $O(\log m + r)$ expected.
Average / best case	Linear in m as deployed.
Auxiliary space $S(\cdot)$	$O(r)$ result.
Volume	118.5 ms per reading at 50,000 edges; at 10^5 edges roughly 240 ms.
Velocity	Gateway readings arrive per sensor per interval; each pays the full scan.
Variety	Sensor values in m or mm/h; geography casts give metre radii.
Measured	4.6 / 26.3 / 118.5 ms at 1k / 10k / 50k edges (96 edges returned). In a controlled run on 50k edges, hand-written SQL with a matching functional index `gist((geom::geography))` took 72 ms against 188 ms for the scan, but the ORM-generated query showed no gain with that index in the harness and the cause was not investigated. No schema change was made.
CI assertion	p95 < 150 ms at 10,000 edges (measured 30 ms).

ALG-16: Hazard coverage (ST_Intersects) (`backend/main.py + PostGIS`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	$O(h \log m + r)$ with the geometry index; $O(hm)$ without.
Average / best case	Flat in m for small polygons.
Auxiliary space $S(\cdot)$	$O(r)$.
Volume	2.4–3.3 ms at 50,000 edges with 20 hazards.
Velocity	Read on each route request; cache per hazard version if needed.
Variety	Polygon hazards and edge-pointer closures are unioned.
Measured	2.4–3.3 ms at 1k, 10k and 50k edges (flat in m, as derived).
CI assertion	< 25 ms at 50,000 edges.

ALG-17: Advisory upsert (last-writer-by-time) (`backend/main.py (_upsert_advisories)`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	$O(r)$ row operations (each an indexed upsert, $O(\log m)$); independent of the total table size.
Average / best case	Same.
Auxiliary space $S(\cdot)$	$O(r)$ parameters.
Volume	11 ms for 96 edges; grows with r only.

Velocity	One call per sensor reading.
Variety	Source strings identify gauges, models or simulated sensors.
Measured	About 11 ms for 96 edges at 1k, 10k and 50k edges (independent of table size).
CI assertion	< 60 ms for 100 edges.

ALG-18: Session tokens (JWT HS256) (`backend/auth_utils.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	$O(\lvert\text{claims}\rvert)$, effectively constant.
Average / best case	Same.
Auxiliary space $S(\cdot)$	$O(1)$.
Volume	Not data-dependent.
Velocity	One verify per authenticated request: 22 µs.
Variety	Claims are small JSON.
Measured	15.4 µs sign, 22.5 µs verify per token.
CI assertion	verify < 100 µs.

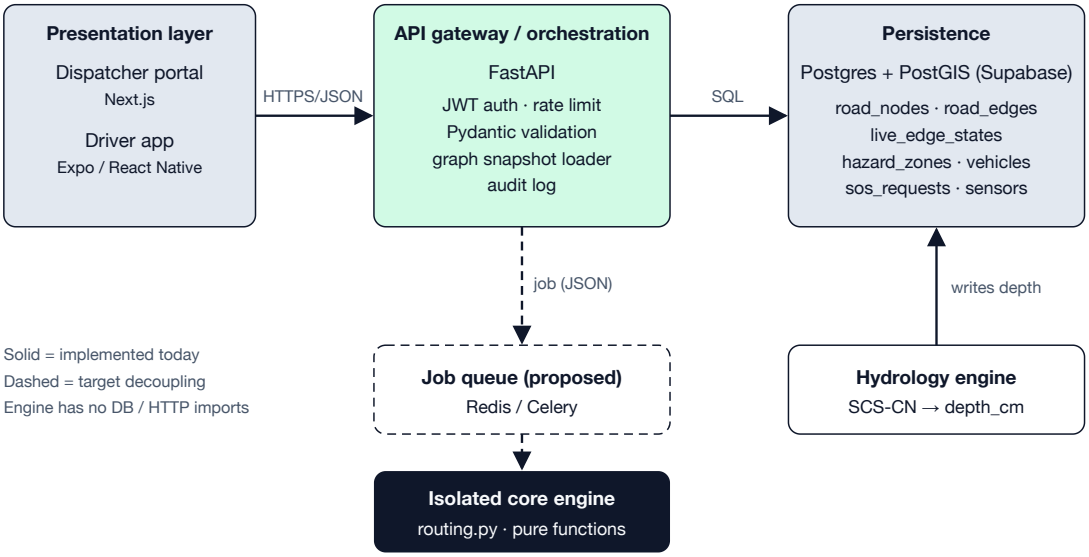
ALG-19: Seed road-network builder (`backend/network_seed.py`)

Measure	Derivation and measurement
Worst-case time $O(\cdot)$	$O(n+m)$ time and space.
Average / best case	Same.
Auxiliary space $S(\cdot)$	$O(n+m)$.
Volume	Pilot: 34 nodes, 61 edges; a city extract would be 10^5 nodes and needs a loader.
Velocity	Run once at seeding.
Variety	Hard-coded names and coordinates.
Measured	0.17 ms for the real seed. City-scale generation was not measured.
CI assertion	seed build < 50 ms; `validate` passes.

Task 4: Decoupled Software Architecture & Service-Layer Integration

Waterfall phase: Design (TSD architecture and API sections). Any change to the contract below would go through a change request (CR), not an ad-hoc edit.

4.1 Component diagram



Current state vs. target. Today `routing.py` is already a pure module: it imports no ORM, HTTP or database code and takes plain dicts and numbers. FastAPI endpoints (`main.py`) load the graph with SQLAlchemy, call it in-process, and format the response. The **target** shown by the dashed boxes moves the call behind a job queue and a worker, so the engine can be scaled and timed out independently of web threads. Boundaries: Presentation never reaches the database; the gateway owns auth, validation and persistence; the engine owns computation only [12].

4.2 Interface data contract

Request: **`POST /api/v1/algorithm/solve`** (JSON, `application/json`, JWT bearer; documented as OpenAPI 3.1 [11]). It generalises the existing `GET /api/v1/routing/navigate` and `GET /api/v1/dispatch/{id}/preview`.

```
{
  "request_id": "7c1e2f0a-3b8e-4a1c-9d57-0a1f2b3c4d5e",
  "algorithm": "td_a_star",
  "start_node": 101,
  "target_node": 482,
  "vehicle": { "vehicle_id": 7, "clearance_cm": 28.0 },
  "options": { "ignore_hazards": false, "time_limit_ms": 250 }
}
```

Constraints: `start_node`, `target_node` integer ≥ 1; `clearance_cm` number, 0 < x ≤ 200; `time_limit_ms` integer 1–5000; unknown fields rejected.

Response certificate (`200 OK`; asynchronous mode returns `202` with `job_id` and the same body on `GET /api/v1/algorithm/jobs/{job_id}`):

```
{
  "request_id": "7c1e2f0a-3b8e-4a1c-9d57-0a1f2b3c4d5e",
  "status": "OK",                       // OK | NO_SAFE_ROUTE | INVALID_NODE | TIMEOUT
  "result": {
    "path": [101, 134, 219, 482],
    "travel_time_s": 412.6,
    "distance_m": 3180.4,
    "segments": [ { "from": 101, "to": 134, "length_m": 820.0, "depth_cm": 6.0,
"passability": "LIGHT_FLOOD" } ],
    "hazards_on_route": []
  },
  "metrics": { "nodes_settled": 348, "edges_relaxed": 1210, "runtime_ms": 14.2, "optimal":
true },
  "flags": { "used_ignore_hazards": false, "truncated": false }
}
```

Errors use RFC 9457 `application/problem+json` [9] with HTTP semantics per RFC 9110 [10]: `422` invalid body, `404` / status `NO_SAFE_ROUTE` when no passable route exists, `401/403` auth, `429` rate limit, `504` `TIMEOUT`.

4.3 Interface contracts for the supporting algorithms

All supporting algorithms sit behind the same boundaries as in the component diagram above: the gateway owns authentication, validation and persistence; pure computation lives in the engine. Each contract below names its layer, its interface, and the request and response shapes.

ALG-02: SCS-CN runoff and ponding depth (`backend/hydrology.py`)

Contract field	Specification
Layer / service boundary	Gateway-triggered computation (admin only), result persisted to `live_edge_states`.
Interface	`POST /api/v1/simulation/rain?rainfall_mm_hr=40&clearance_cm=28`
Request	Query parameters: `rainfall_mm_hr` 0–250 (required), `clearance_cm` 0 < x ≤ 200; admin only; duration comes from configuration.
Response	{ network GeoJSON with per-edge depth_cm and passability } // same shape the map renders

ALG-03: Passability classification and hazard blocking (`backend/routing.py`)

Contract field	Specification
Layer / service boundary	Pure functions in the engine; the gateway uses the labels in responses.
Interface	Embedded in `GET /api/v1/network/state` and `GET /api/v1/routing/navigate` responses.
Request	`clearance_cm` query parameter, 0 < x ≤ 200.
Response	"passability": "PASSABLE | LIGHT_FLOOD | SUBMERGED" per segment

ALG-04: Turn-by-turn instruction generation (`backend/routing.py`)

Contract field	Specification
Layer / service boundary	Gateway post-processing of the engine's path.

Interface	Part of `GET /api/v1/routing/navigate`
Request	`start_node`, `target_node`, `clearance_cm`.
Response	"instructions": [{ "manoeuvre": "TURN_LEFT", "road_name": "...", "distance_m": 410.0, "text": "Turn left onto ..." }]

ALG-05: Dispatch preview (one search per vehicle) (`backend/main.py`)

Contract field	Specification
Layer / service boundary	Gateway orchestration calling the isolated engine K times.
Interface	`GET /api/v1/dispatch/{sos_id}/preview?vehicle_id=7`
Request	`sos_id` in the path; optional `vehicle_id`.
Response	{ "units": [{ "vehicle_id": 7, "travel_time_s": 412.6, "route": { ... }, "detail": null }] } // 409 no position, 422 no safe route

ALG-06: Nearest-node snap (`backend/main.py (_nearest_node)`)

Contract field	Specification
Layer / service boundary	Gateway helper before the engine call.
Interface	Internal; used by the dispatch preview and telemetry handling.
Request	latitude, longitude.
Response	node_id (integer).

ALG-07: Flood-state resolver (CR-004 priority rule) (`backend/flood_state.py`)

Contract field	Specification
Layer / service boundary	Pure core function; the gateway loads inputs from `edge_advisories`, hazards and NOAH data.
Interface	`GET /api/v1/flood-state`
Request	Optional bounding area; authenticated.
Response	{ "generated_at": "...", "raining": true, "fresh_window_minutes": 30, "edges": [{ "edge_id": 12, "basis": "LIVE_ADVISORY", "source": "SENSOR:RIVER-01", "advisory": true, "closes_road": false }] }

ALG-08: Report urgency scoring and ranking (`backend/reports.py`)

Contract field	Specification
Layer / service boundary	Gateway computation over rows already loaded from the database.
Interface	`GET /api/v1/reports?status=PENDING&limit=50`
Request	`status`, `limit` 1–200.
Response	[{ "report_id": "uuid", "depth_level": "KNEE", "source": "PHOTO", "urgency": 3.41, "duplicate_count": 2 }]

ALG-09: Duplicate-neighbour map for the moderation queue (`backend/main.py (_neighbour_map)`)

Contract field	Specification
Layer / service boundary	Gateway computation over rows already in memory.
Interface	Part of `GET /api/v1/reports`
Request	As ALG-08.
Response	"duplicate_count": 2, "nearest_duplicate_id": "uuid"

ALG-10: Sensor/report mismatch check and haversine distance (`backend/reports.py`)

Contract field	Specification
Layer / service boundary	Gateway computation; gauges loaded from the database.
Interface	`GET /api/v1/reports/{report_id}/nearest-sensor` and a flag on each report
Request	`report_id`; admin only.
Response	"sensor_mismatch": true

ALG-11: Evidence checks: digest, image type, EXIF, polygon, reporter hash (`backend/reports.py`)

Contract field	Specification
Layer / service boundary	Gateway, inside the report intake path.
Interface	`POST /api/v1/reports` (multipart upload)
Request	Form fields: `photo` (JPEG), `latitude`, `longitude`, `depth_level`, optional `narrative`, `barangay`, `captured_at`, `source`; size and type limits enforced.
Response	{ "photo_sha256": "...", "exif": { "captured_at": "...", "gps_in_pilot_area": true }, "location_plausible": true }

ALG-12: Tamper-evident audit hash chain (`backend/audit.py`)

Contract field	Specification
Layer / service boundary	Gateway, in the same transaction as the audited action.
Interface	`GET /api/v1/audit/verify`
Request	Admin only.
Response	{ "valid": true, "checked": 1542, "broken_at": null }

ALG-13: Windowed rate limiter (`backend/rate_limit.py`)

Contract field	Specification
Layer / service boundary	Gateway dependency on selected routes.
Interface	FastAPI dependency `rate_limit(name, limit, window)`
Request	Request headers (client id).
Response	429 problem document when exceeded.

ALG-14: SSE event replay and framing (`backend/events.py`)

Contract field	Specification
Layer / service boundary	Gateway streaming endpoints.
Interface	`GET /api/v1/hazards/stream`, `/vehicles/stream`, `/sos/stream`
Request	`Last-Event-ID` header or `after` query parameter.
Response	`text/event-stream` frames: `id: 42 / event: hazard.published / data: {...}`

ALG-15: Sensor impact-radius edge marking (ST_DWithin) (`backend/main.py + PostGIS`)

Contract field	Specification
Layer / service boundary	Gateway (admin or sensor-gateway credential) with the spatial query in the database.
Interface	`POST /api/v1/sensors/readings`
Request	{ "sensor_id": "RIVER-01", "value": 4.2, "unit": "m", "recorded_at": "2026-11-04T10:00:00Z" }
Response	201 { "reading_id": 91, "breach": true, "value": 4.2 } // 404 unknown sensor, 409 disabled, 422 unit or future timestamp

ALG-16: Hazard coverage (ST_Intersects) (`backend/main.py + PostGIS`)

Contract field	Specification
Layer / service boundary	Gateway query feeding the engine's blocked-edge set.
Interface	Internal to `GET /api/v1/routing/navigate` and `GET /api/v1/hazards`
Request	None beyond the route query.
Response	"hazards_on_route": [{ "hazard_id": 3, "obstruction_level": "LEVEL_2" }]

ALG-17: Advisory upsert (last-writer-by-time) (`backend/main.py (_upsert_advisories)`)

Contract field	Specification
Layer / service boundary	Gateway persistence step inside the reading transaction.
Interface	Internal to `POST /api/v1/sensors/readings`
Request	edge ids, source, kind, flagged, as_of.
Response	None (rows committed with the reading).

ALG-18: Session tokens (JWT HS256) (`backend/auth_utils.py`)

Contract field	Specification
Layer / service boundary	Gateway authentication.
Interface	`POST /api/v1/auth/login` issues the token; every protected route verifies it.
Request	{ "email": "...", "password": "..." }
Response	{ "access_token": "...", "token_type": "bearer" }

ALG-19: Seed road-network builder (`backend/network_seed.py`)

Contract field	Specification
Layer / service boundary	Offline seeding script and start-up seed.
Interface	`python -m network_seed`
Request	Optional dry-run flag.
Response	Counts of nodes and edges inserted.

Task 5: Implementation Work Packages & Empirical Validation Plan

Waterfall framing. The template calls this task an "Agile backlog" and asks for user stories. MetroFlow is built under waterfall, so we keep the required story wording but treat each story as a *requirement statement traced to the FSD/TSD*, and the ticket as a *work package* with a Verification-phase exit test. There are no sprints: scope is fixed by the approved FSD/TSD, and any change goes through a change request (the project's CR-001 to CR-004 process) rather than backlog re-prioritisation.

Item	Traces to	Phase gate
Story 1 (dispatcher)	FSD-FR-089 (assign a unit and preview the flood-safe route); TSD ADR-004; VAL-022	Design approved; Verification test below
Story 2 (driver)	FSD-FR-057 (turn-by-turn instructions); TSD §2.6 hazard-blocking rule; TC-024	Design approved; Verification test below
DEV-ALGO-101	Both stories; PMP Testing Management: "hydrology and routing edge cases covered"	Implementation (engine built) → Verification (benchmark added) → Acceptance

5.1 User stories (requirement statements)

1. **As a** dispatcher, **I want** the system to run the flood-aware A* search from every available rescue vehicle to a victim's SOS location using each vehicle's own clearance, **so that** I can assign the unit with the lowest safe travel time and never send a unit into water it cannot cross.
2. **As a** rescue driver, **I want** the route calculation to re-run on the latest water depths and hazards, and tell me clearly when no safe route exists, **so that** I am rerouted around newly flooded streets instead of being routed into one.

5.2 Implementation work package & benchmarking ticket

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-101
Module / Service	`backend/routing.py` (engine), `backend/tests/test_routing.py` (correctness) and `backend/benchmarks/bench_algorithms.py` (benchmark harness for every algorithm, written and run including the database-bound ones; CI wiring remains); API wiring in `backend/main.py`.
Associated Algorithm	Clearance-constrained, depth-penalized A* with min-heap (`td_a_star_search`).

Edge-Case & Security Defenses	• Unknown `start`/`target` node → `INVALID_NODE`, not an exception. • `start == target` → `[start]`, $T = 0$. • Empty graph, or disconnected components (target unreachable) → `NO_SAFE_ROUTE`, `inf`. • Depth $\geq$ clearance on every cut edge → no route (404). Depth exactly equal to clearance is blocked. • Zero or negative speed, NaN / negative depth → reject at the gateway; speed floor 0.5 m/s in the engine prevents division by zero. • Clearance bound \(0 • Rate limiting and JWT auth at the gateway; per-query time limit to stop abusive or pathological inputs.
Concrete Implementation Steps	1. Build the adjacency map from `road_edges` joined with `live_edge_states` (length, speed, depth), excluding hazard-closed edges. 2. Compute $v_{\max}$ and the haversine heuristic for the query target. 3. Initialise `g`, `parent`, `settled` and the min-heap with $(h(s), 0, s)$. 4. Pop, skip stale/settled entries, settle, and return on the target. 5. Relax out-edges: apply the clearance cut-off, depth penalty and speed floor; push improved neighbours. 6. Reconstruct the path; return the certificate with metrics (nodes settled, runtime). 7. Add a Dijkstra reference implementation in tests for the optimality check. 8. Add the benchmark test and wire it into the CI gate, which runs in the Verification phase.
Automated Benchmarking Assertion	Measured on the development machine, then set at about 5× the measured value for CI (Sections 3.5 and 3.6): • `assert astar_cost == pytest.approx(dijkstra_cost)` on random grid queries: **0 mismatches in 120 of 120** measured. • `assert p95_runtime_ms < 50` on a grid with N = 5,041 nodes (19,880 edges): measured p95 10.4 ms (median 3.4 ms). The earlier 250 ms target was too loose to catch regressions. • `assert nodes_settled_astar <= nodes_settled_dijkstra`: **true on all 120 queries**, 33–43% fewer on average. (Wall time is *not* asserted to beat Dijkstra: it does not.) • `assert route is None` (empty path, `inf`) when every edge on the only path has depth ≥ clearance. • Supporting algorithms (each ticket in 5.3 has its own assertion): SCS-CN over 100,000 nodes < 300 ms; urgency scoring and sort of 100,000 reports < 500 ms; audit-chain verify of 100,000 entries < 2.5 s with tampering reported at the modified `log_id`; 10,000-node turn-by-turn < 30 ms; 1,000-gauge mismatch check < 2 ms per call; flood-state resolver over 100,000 edges < 1.5 s; duplicate-neighbour map of 200 reports < 150 ms; nearest-node snap over 10,000 nodes < 25 ms; rate-limit upsert p95 < 10 ms and SSE replay p95 < 20 ms at 500,000 rows (local database); impact-radius query p95 < 150 ms at 10,000 edges; a test that `classify_passability` and the A* clearance cut-off never disagree.

The thresholds come from one run on one machine and are not yet enforced in CI; wiring them into the Verification gate is the remaining part of this work package. This follows the project's testing exit criteria (unit tests pass, routing edge cases covered) before the work package is accepted.

5.3 User stories and work packages for the supporting algorithms

As in 5.1–5.2, each story is a requirement statement traced to the FSD/TSD, and each ticket is a work package whose exit test is its benchmark assertion. The code for each already exists; the remaining item for every ticket is wiring its assertion into the CI gate.

ALG-02: SCS-CN runoff and ponding depth (`backend/hydrology.py`)

As a dispatcher, **I want** the system to convert a rainfall input into street flood depths on every road, **so that** the route search and the map start from current water levels.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-102
Module / Service	`backend/hydrology.py`
Associated Algorithm	ALG-02: SCS-CN runoff and ponding depth
Traces to	BR-002 / FSD-FR-006 to 010; TSD ADR-005; test TTC-003.
Edge-Case & Security Defenses	$P \le 0$ or $P \le I_a \rightarrow 0$; elevation clamp; result clamped at 0; admin-only trigger and per-minute rate limit at the gateway.
Concrete Implementation Steps	1. Compute P, S, I_a. 2. Return 0 when $I_a \ge P$. 3. Compute Q. 4. Apply the elevation factor and α. 5. Write depth to each node's edges.
Automated Benchmarking Assertion	`assert t(100_000 nodes) < 300 ms` (measured 60 ms); `assert Q(P1) <= Q(P2)` for $P1 < P2$; `assert Q(0) == 0`.

ALG-03: Passability classification and hazard blocking (`backend/routing.py`)

As a driver, **I want** each road segment labelled passable, light flood or submerged for my vehicle's clearance, **so that** the map colours and the route always agree.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-103
Module / Service	`backend/routing.py`
Associated Algorithm	ALG-03: Passability classification and hazard blocking
Traces to	FSD-FR-011 to 017 (routing and map); BRULE-008; TSD §2.6.
Edge-Case & Security Defenses	Inclusive threshold at $d = c$; unknown obstruction level does not block; clearance bound validated at the API.
Concrete Implementation Steps	1. Implement the two rules in one module. 2. Have the router and the map call them (or assert equivalence). 3. Add a sweep test across d and c.
Automated Benchmarking Assertion	`assert t(10**6 calls) < 1.5 s` (measured 0.23 s); sweep equivalence between `classify` and the A* cut-off.

ALG-04: Turn-by-turn instruction generation (`backend/routing.py`)

As a rescue driver, **I want** spoken turn-by-turn steps that skip trivial bends, **so that** I can follow the route without watching the map.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-104
Module / Service	`backend/routing.py`
Associated Algorithm	ALG-04: Turn-by-turn instruction generation

Traces to	FSD-FR-057; test TC-024.
Edge-Case & Security Defenses	Empty path → no steps; single node → ARRIVE only; missing coordinate → DEPART + ARRIVE.
Concrete Implementation Steps	1. Compute hop bearings. 2. Classify heading changes. 3. Merge straight runs. 4. Sum distances per step. 5. Render text.
Automated Benchmarking Assertion	`assert t(10_000 nodes) < 30 ms` (measured 6.1 ms); `assert sum(step.distance) == path_length`.

ALG-05: Dispatch preview (one search per vehicle) (`backend/main.py`)

As a dispatcher, **I want** a route preview for each available unit using its own clearance, **so that** I can assign the unit with the lowest safe travel time.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-105
Module / Service	`backend/main.py`
Associated Algorithm	ALG-05: Dispatch preview (one search per vehicle)
Traces to	FSD-FR-089; test VAL-022; TSD ADR-004.
Edge-Case & Security Defenses	Missing vehicle position → 409; no safe route → 422; unavailable unit cannot be assigned (BRULE-011).
Concrete Implementation Steps	1. Snap SOS and each unit to nodes. 2. Run A* per unit with its clearance. 3. Collect results and reasons. 4. Sort by time.
Automated Benchmarking Assertion	`assert t(K=10, N=5041) < 200 ms` (measured 39 ms); each result equals the Dijkstra oracle for that clearance.

ALG-06: Nearest-node snap (`backend/main.py (_nearest_node)`)

As a dispatcher, **I want** a unit's GPS position and an SOS location snapped to the nearest road node, **so that** the route search can start and end on the graph.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-106
Module / Service	`backend/main.py (_nearest_node)`
Associated Algorithm	ALG-06: Nearest-node snap
Traces to	FSD-FR-089 (a unit joins the graph by snapping).
Edge-Case & Security Defenses	Empty node set raises; coordinates validated by the API.
Concrete Implementation Steps	1. Iterate nodes. 2. Keep the minimum haversine. 3. Return its id. 4. (Planned) replace with an indexed lookup at city scale.
Automated Benchmarking Assertion	`assert t(10_000 nodes) < 25 ms` (measured 5 ms); result equals a brute-force oracle.

ALG-07: Flood-state resolver (CR-004 priority rule) (`backend/flood_state.py`)

As a dispatcher, **I want** each road segment to show which source flagged it and how fresh that source is, **so that** I can judge advisories without mistaking a model for a measurement.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-107
Module / Service	`backend/flood_state.py`
Associated Algorithm	ALG-07: Flood-state resolver (CR-004 priority rule)
Traces to	CR-004 §3.3; BRULE-012, BRULE-016.
Edge-Case & Security Defenses	Future timestamps beyond 5 min are ignored; stale flagged readings are noted; unknown NOAH class gets penalty 1.0; no data is NONE, not safe.
Concrete Implementation Steps	1. Apply the verified-hazard rule. 2. Filter fresh flagged observations and take the best. 3. Apply NOAH only when raining. 4. Else return NONE with coverage flags.
Automated Benchmarking Assertion	`assert t(100_000 edges) < 1.5 s` (measured 0.28 s); property test: `closes_road` implies a blocking verified hazard. Status: implemented and served; routing does not read it yet.

ALG-08: Report urgency scoring and ranking (`backend/reports.py`)

As a moderator, **I want** pending reports ranked by severity, source reliability, corroboration and freshness, **so that** the most urgent evidence is reviewed first.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-108
Module / Service	`backend/reports.py`
Associated Algorithm	ALG-08: Report urgency scoring and ranking
Traces to	TSD §2.8; moderation desk requirements (FSD ID to confirm).
Edge-Case & Security Defenses	Unknown source falls back to CALL; negative age or neighbour count is clamped; an unknown depth level raises (callers pass the validated enum).
Concrete Implementation Steps	1. Look up severity and source weight. 2. Apply the corroboration cap. 3. Decay by age. 4. Sort descending.
Automated Benchmarking Assertion	`assert t(100_000) < 500 ms` (measured 101 ms); `assert U(a2) < U(a1)` for $a2 > a1$.

ALG-09: Duplicate-neighbour map for the moderation queue (`backend/main.py (_neighbour_map)`)

As a moderator, **I want** reports near each other in space and time flagged as likely duplicates, **so that** I can approve one marker instead of several.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-109
Module / Service	`backend/main.py (_neighbour_map)`
Associated Algorithm	ALG-09: Duplicate-neighbour map for the moderation queue

Traces to	Duplicate hint on the moderation queue (FSD ID to confirm).
Edge-Case & Security Defenses	Reports without a location or not pending get (0, none); the page cap of 200 bounds the $O(k^2)$ cost.
Concrete Implementation Steps	1. Collect pending located reports. 2. Compare each pair within the time window. 3. Count within 150 m and track the nearest. 4. (Planned) grid bucketing.
Automated Benchmarking Assertion	`assert t(200 reports) < 150 ms` (measured 34 ms); `assert symmetric(counts)`.

ALG-10: Sensor/report mismatch check and haversine distance (`backend/reports.py`)

As a moderator, **I want** a report flagged when the nearest rain gauge shows no breach, **so that** I look harder at claims that sensors contradict.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-110
Module / Service	`backend/reports.py`
Associated Algorithm	ALG-10: Sensor/report mismatch check and haversine distance
Traces to	FSD-FR-026, FSD-FR-079.
Edge-Case & Security Defenses	No coordinates, no gauge in range, or a gauge without a reading: false (absence of data is not a contradiction).
Concrete Implementation Steps	1. Filter gauges within 5 km. 2. Keep the nearest. 3. Compare reading and breach state.
Automated Benchmarking Assertion	`assert t(g=1000) < 2 ms` (measured 0.41 ms); `assert haversine(p, p) == 0`.

ALG-11: Evidence checks: digest, image type, EXIF, polygon, reporter hash (`backend/reports.py`)

As a moderator, **I want** each photo's integrity hash, capture time and camera position extracted automatically, **so that** I can check the photo matches where and when the report claims.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-111
Module / Service	`backend/reports.py`
Associated Algorithm	ALG-11: Evidence checks: digest, image type, EXIF, polygon, reporter hash
Traces to	FSD-FR-078 (EXIF); BR-011 (reporter privacy).
Edge-Case & Security Defenses	Oversize or wrong-type upload rejected; malformed EXIF returns empty; GPS outside the pilot area flagged, not rejected.
Concrete Implementation Steps	1. Sniff the type. 2. Hash the bytes. 3. Parse APP1/TIFF tags. 4. Check bounds. 5. Build the polygon on approval.
Automated Benchmarking Assertion	`assert t(digest 5 MB) < 50 ms` (measured 2.2 ms); `assert read_exif(truncated) == {}`.

ALG-12: Tamper-evident audit hash chain (`backend/audit.py`)

As an administrator, **I want** to verify that the audit log has not been edited or reordered, **so that** dispatch and user-management actions stay attributable.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-112
Module / Service	`backend/audit.py`
Associated Algorithm	ALG-12: Tamper-evident audit hash chain
Traces to	Audit log requirement (FSD ID to confirm); FSD-FR-048/049 audited actions.
Edge-Case & Security Defenses	Unparseable payload counts as broken at that entry; first record has no predecessor; empty log is valid.
Concrete Implementation Steps	1. Canonicalise the record. 2. Hash with the previous hash. 3. Store. 4. On verify, recompute in order and compare.
Automated Benchmarking Assertion	`assert t(100_000) < 2.5 s` (measured 0.47 s); `assert broken_at == tampered.log_id`.

ALG-13: Windowed rate limiter (`backend/rate_limit.py`)

As an operator, **I want** login, registration, SOS and report endpoints rate limited per client, **so that** abuse cannot overwhelm the service.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-113
Module / Service	`backend/rate_limit.py`
Associated Algorithm	ALG-13: Windowed rate limiter
Traces to	TSD LIM-006.
Edge-Case & Security Defenses	Limit ≤ 0 disables; header spoofing is a documented limit behind non-replacing proxies.
Concrete Implementation Steps	1. Derive the bucket. 2. Run the atomic upsert. 3. Compare with the limit. 4. Return 429 when over.
Automated Benchmarking Assertion	`assert p95(consume) < 10 ms` (measured 1.4 ms local); test: the $(\lambda + 1)$th call in a window is rejected.

ALG-14: SSE event replay and framing (`backend/events.py`)

As a dispatcher, **I want** the map to resume live updates after a dropped connection without missing or repeating events, **so that** the radar always matches reality.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-114
Module / Service	`backend/events.py`
Associated Algorithm	ALG-14: SSE event replay and framing

Traces to	FSD-FR-086, BR-018; ADR-024.
Edge-Case & Security Defenses	Malformed cursor → error; fresh subscribers get a snapshot; limit caps each replay.
Concrete Implementation Steps	1. Append an event in the same transaction as the change. 2. On connect, replay after the cursor. 3. Then tail new ids. 4. Frame and send.
Automated Benchmarking Assertion	`assert p95(replay) < 20 ms` at 500k rows (measured 2.5 ms p95); test: reconnect with a cursor returns no duplicates.

ALG-15: Sensor impact-radius edge marking (ST_DWithin) (`backend/main.py + PostGIS`)

As a dispatcher, **I want** a river-gauge breach to flag the roads within its impact radius as advisory, **so that** I see which streets may be affected before confirming a hazard.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-115
Module / Service	`backend/main.py + PostGIS`
Associated Algorithm	ALG-15: Sensor impact-radius edge marking (ST_DWithin)
Traces to	FSD-FR-090, FSD-FR-091; BRULE-012.
Edge-Case & Security Defenses	Unit mismatch, disabled sensor, future timestamp and NaN/inf values rejected; a breach is advisory until a dispatcher confirms.
Concrete Implementation Steps	1. Validate the reading. 2. Compare to the threshold. 3. Query edges in radius. 4. Upsert advisories. 5. Audit on breach.
Automated Benchmarking Assertion	`assert p95(query) < 150 ms` at 10,000 edges (measured 30 ms); `assert plan uses an index` (currently fails: sequential scan).

ALG-16: Hazard coverage (ST_Intersects) (`backend/main.py + PostGIS`)

As a driver, **I want** routes to avoid verified hazards and list those near my route, **so that** I am never sent through a closed road.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-116
Module / Service	`backend/main.py + PostGIS`
Associated Algorithm	ALG-16: Hazard coverage (ST_Intersects)
Traces to	TSD §2.6; BRULE-008.
Edge-Case & Security Defenses	Inactive hazards ignored; closures without geometry still reroute; ignore_hazards only for the explicit unsafe alternative.
Concrete Implementation Steps	1. Load active hazards. 2. Spatial-join to edges. 3. Union direct edge ids. 4. Pass blocking edges to the engine.
Automated Benchmarking Assertion	`assert t(50_000 edges, 20 hazards) < 25 ms` (measured 3 ms).

ALG-17: Advisory upsert (last-writer-by-time) (`backend/main.py (_upsert_advisories)`)

As a dispatcher, **I want** the newest reading from each gauge recorded on the roads it covers without older readings overwriting it, **so that** advisories reflect the latest data.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-117
Module / Service	`backend/main.py (_upsert_advisories)`
Associated Algorithm	ALG-17: Advisory upsert (last-writer-by-time)
Traces to	CR-004; FSD-FR-091.
Edge-Case & Security Defenses	Empty edge list returns immediately; out-of-order readings do not overwrite newer ones.
Concrete Implementation Steps	1. Build one multi-row insert. 2. Conflict on (edge, source). 3. Update only when the new time is not older.
Automated Benchmarking Assertion	`assert t(100 edges) < 60 ms` (measured 11 ms); test: an older `as_of` leaves the row unchanged.

ALG-18: Session tokens (JWT HS256) (`backend/auth_utils.py`)

As a user, **I want** to sign in once and have my session verified on each request, **so that** the service stays stateless and my role is enforced.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-118
Module / Service	`backend/auth_utils.py`
Associated Algorithm	ALG-18: Session tokens (JWT HS256)
Traces to	TSD ADR-021.
Edge-Case & Security Defenses	Expired, tampered or wrongly-algorithmed tokens return 401; the secret is never committed.
Concrete Implementation Steps	1. Verify credentials. 2. Sign claims with an expiry. 3. Verify on every request. 4. Enforce roles.
Automated Benchmarking Assertion	`assert t(verify) < 100 µs` (measured 22 µs); test: a tampered token is rejected.

ALG-19: Seed road-network builder (`backend/network_seed.py`)

As a developer, **I want** a reproducible seed graph, **so that** tests and demos run on the same network.

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-119
Module / Service	`backend/network_seed.py`
Associated Algorithm	ALG-19: Seed road-network builder
Traces to	TSD seed section; ALGORITHMS_DEMO.md walkthrough.

Edge-Case & Security Defenses	Validation rejects dangling edges and duplicate ids.
Concrete Implementation Steps	1. Build lattice nodes. 2. Add edges in both directions. 3. Validate. 4. Apply to the database.
Automated Benchmarking Assertion	`assert t(build) < 50 ms` (measured 0.17 ms); `assert validate(nodes, edges).`

Task 6: Academic Review Questions & Defense Checkpoints

Q1. Why must heavy algorithmic processing be decoupled from database ORMs and web request threads?

(a) **Responsiveness:** a CPU-bound search holds a worker thread for its whole run; with a bounded thread pool, K concurrent searches starve unrelated requests (login, map reads) [12]. Moving them to a worker pool behind a queue [15] bounds request latency and allows independent scaling. (b) **Testability and correctness:** a pure function over plain numbers can be unit-tested and fuzzed against a reference (Dijkstra) without a database; ORM lazy loading inside the loop would add a hidden query per edge, turning $O(m \log n)$ compute into $O(m)$ database round-trips. (c) **Consistency:** the engine should read an immutable snapshot so concurrent depth writes cannot change costs mid-search, which would break the optimality argument. (d) **Replaceability:** the same engine can be called by the dispatch preview, the driver app and batch jobs. In a waterfall plan this also keeps the design stable: the contract in Task 4 is fixed at the Design gate, so the engine can be re-implemented without reopening the API.

Q2. If the algorithm has exponential worst-case complexity, what keeps it viable?

Our core engine is not exponential: it is $O(m \log n)$ (Task 3). The exponential algorithm is the *baseline* (path enumeration). Had we extended the problem to a *multi-criteria* or *capacity-constrained* variant, which is NP-hard, we would use pruning by the admissible bound f [2], [3], dominance pruning, and an ε-bounded weighted A* ($f = g + (1 + \varepsilon)h$) that guarantees a cost within $(1 + \varepsilon)$ of optimal, plus the per-request `time_limit_ms` returning the best route found with `truncated=true`.

Q3. Exact vs. approximate / heuristic algorithms in production?

An exact algorithm guarantees the optimal answer and a checkable certificate, at a cost that may grow quickly. A heuristic or approximate algorithm trades a bounded or unbounded loss in quality for speed [1]. Our A* is **exact** because h is admissible and consistent [2], [3]; "heuristic" describes how it is guided, not that its answer is approximate. In a rescue setting a wrong (non-optimal) route has a safety cost, so we keep the exact search and use an approximate variant only as a time-limited fallback. The hydrology model, by contrast, is an approximation (SCS-CN, a decision aid, not a hydrodynamic model) [8], and we label it as such.

Q4. Empirical benchmarking vs. asymptotic analysis?

Asymptotic analysis is done *before* implementation, in the Design phase, on a model of computation, and states how cost grows with n, m while ignoring constants, hardware and the interpreter [1]. Empirical profiling is done *after* implementation, in the Verification phase, on real code and data, and measures constants, cache and Python overhead, and the average case that theory cannot give in closed form. We use both: Task 3 predicts $O(m \log n)$; the benchmark harness (Tasks 3 and 5) checks the growth trend (runtime vs. N) and enforces an absolute budget (p95 < 50 ms at N = 5,041). Neither replaces the other: a

good bound with a slow constant can still miss a latency budget (our A* settles fewer nodes than Dijkstra yet runs slower in Python), and a fast run on one dataset proves nothing about the worst case.

Q5. Cold start / data sparsity (if ML or preference matrices are used)?

The routing engine uses no learned model or user-preference matrix, so the classic cold-start problem does not apply. The analogous risk is **sparse live data**: few sensors and reports at launch, so many edges have no measured depth. Mitigation: default unknown edges to the hydrology engine's modelled depth [8], treat sensor and verified-report values as overrides, and surface edges without recent data as lower-confidence rather than silently safe. If we later add a learned depth predictor, we would seed it from the physical model and historic flood records.

Q6. How are edge-case inputs handled without crashing the service?

Every degenerate input has a defined return value rather than an exception: unknown nodes → `INVALID_NODE`; `s = t` → zero-length path; **disconnected subgraphs** or fully flooded corridors → the frontier empties and the search returns `NO_SAFE_ROUTE` (the termination certificate in line 18); **cycles** are harmless because `settled` makes every node expand once [1]; the speed floor removes division by zero. No singular-matrix case arises, as the engine uses no linear algebra. Inputs are validated at the gateway (Pydantic, bounded ranges) and errors are returned as RFC 9457 problem documents [9], so a bad request cannot take down the worker.

6.7 Answers by algorithm (Q2, Q3, Q5, Q6)

Q1 and Q4 apply to the whole system. For the others, each algorithm answers separately:

Algorithm	Q2: exponential blow-up?	Q3: exact or approximate?	Q5: cold start / sparse data	Q6: edge cases
ALG-01: Flood-aware A*	No: $O(m \log n)$; baseline is exponential	**Exact** (admissible, consistent heuristic)	Defaults to modelled depth when edges lack readings	Defined statuses; unreachable target returns NO_SAFE_ROUTE
ALG-02: SCS-CN runoff and ponding depth	Not exponential: $O(1)$ per node.	A closed-form **approximation** of flooding (a decision aid), labelled as such.	No learning. Sparse gauges are handled by using this modelled depth as the default (Q5).	Zero or negative rain returns 0; no division by zero ($CN > 0$).
ALG-03: Passability classification and hazard blocking	Not exponential: $O(1)$.	**Exact** rule evaluation.	No data dependence.	Depth equal to clearance is submerged; unknown level is non-blocking, never an exception.
ALG-04: Turn-by-turn instruction generation	Not exponential: $O(k)$.	**Exact** given the thresholds.	No data dependence.	Empty or one-node path and missing coordinates return a defined fallback.
ALG-05: Dispatch preview (one search per vehicle)	Polynomial: $K \times O(m \log n)$.	**Exact** per vehicle.	No learning; a unit without telemetry is reported, not guessed.	Missing position, no route and unavailable unit return defined statuses.
ALG-06: Nearest-node snap	Polynomial; no blow-up. Index recommended for scale.	**Exact**.	No data dependence.	Empty graph raises `ValueError`; the gateway checks nodes exist first.

ALG-07: Flood-state resolver (CR-004 priority rule)	Polynomial: $O(m\bar{o})$.	**Exact** rule evaluation over inputs that are themselves estimates.	Sparse coverage is explicit: NONE with `has_live_data=false`, never "safe".	Stale, future-dated and missing inputs have defined outcomes; pure function cannot crash on I/O.
ALG-08: Report urgency scoring and ranking	No blow-up: $O(k \log k)$.	**Exact** ordering under a hand-set scoring model (the weights are judgement, not learned).	No learning: weights are fixed, so there is no cold start; a new source type defaults to CALL.	Clamped inputs; empty queue returns empty.
ALG-09: Duplicate-neighbour map for the moderation queue	Polynomial but quadratic; mitigated by the 200-row cap and, if needed, by grid bucketing.	**Exact** neighbour counts.	No learning; sparse data simply gives counts of zero.	Missing coordinates and non-pending rows are skipped without error.
ALG-10: Sensor/report mismatch check and haversine distance	No blow-up: $O(g)$.	**Exact**; the answer is advisory.	Sparse gauges: out-of-range or reading-less gauges produce no flag.	All degenerate cases return false.
ALG-11: Evidence checks: digest, image type, EXIF, polygon, reporter hash	No blow-up: linear in bytes or metadata entries.	**Exact** parsing; the plausibility flag is advisory.	Photos without EXIF still work: the checks are optional evidence.	Truncated, non-JPEG or corrupt data returns "no EXIF" instead of rejecting the report.
ALG-12: Tamper-evident audit hash chain	No blow-up: $O(L)$.	**Exact** detection of in-place edits (not of full-database compromise).	No learning.	Corrupt payloads and empty logs return defined results.
ALG-13: Windowed rate limiter	No blow-up.	**Exact** count within a window; boundary burst is a known approximation.	Not applicable.	Database error handling: the limiter must not take the endpoint down (see module).
ALG-14: SSE event replay and framing	No blow-up.	**Exact** ordered delivery.	No learning.	Bad cursors are rejected; empty replays return nothing.
ALG-15: Sensor impact-radius edge marking (ST_DWithin)	No blow-up, but linear in m as deployed; add the functional index.	**Exact** radius selection.	Sparse sensors: edges outside every radius have no live data and are reported as such (ALG-07).	Invalid readings are rejected with 4xx before any write.
ALG-16: Hazard coverage (ST_Intersects)	No blow-up with the index.	**Exact** geometric intersection.	No hazards means an empty mapping, not an error.	Hazards without polygons are handled through the edge pointer.
ALG-17: Advisory upsert (last-writer-by-time)	No blow-up.	**Exact**.	No learning.	Empty input and stale input are no-ops.

ALG-18: Session tokens (JWT HS256)	No blow-up.	**Exact** verification.	No learning.	Any decode error returns 401, not a crash.
ALG-19: Seed road-network builder	No blow-up.	**Exact**, deterministic.	Not applicable.	Dangling or duplicate entries fail validation at seeding, not at runtime.

<!-- AI Assistance: Claude wrote the benchmark harness (`backend/benchmarks/bench_algorithms.py`), ran it (including on a throwaway local PostGIS), and drafted the per-algorithm sections of Tasks 1–6 from the code and the measured output. All timings are from one run on one machine, the complexity entries are our own reading of the code, and statements marked "FSD ID to confirm" need checking against the FSD. The team should rerun the harness before citing numbers and check any algorithm it wants to defend. -->

References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
2. P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968. (Seminal source, pre-2021.)
3. R. Dechter and J. Pearl, "Generalized best-first search strategies and the optimality of A*," *J. ACM*, vol. 32, no. 3, pp. 505–536, 1985. (Seminal source, pre-2021.)
4. E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. (Seminal source, pre-2021.)
5. H. Bast *et al.*, "Route planning in transportation networks," in *Algorithm Engineering*, LNCS vol. 9220. Cham, Switzerland: Springer, 2016, pp. 19–80.
6. S. Tsang and D. M. Scott, "An integrated approach to modeling the impact of floods on emergency services: A case study of Calgary, Alberta," *J. Transport Geography*, vol. 86, 2020.
7. M. Pregnolato, A. Ford, S. M. Wilkinson, and R. J. Dawson, "The impact of flooding on road transport: A depth-disruption function," *Transp. Res. D, Transp. Environ.*, vol. 55, pp. 67–81, 2017.
8. USDA Natural Resources Conservation Service, "Estimation of direct runoff from storm rainfall," in *National Engineering Handbook, Part 630 Hydrology*, ch. 10, 2004.
9. M. Nottingham, E. Wilde, and S. Dalal, "Problem details for HTTP APIs," IETF, RFC 9457, Jul. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9457
10. R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP semantics," IETF, RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110
11. OpenAPI Initiative, "OpenAPI Specification v3.1.0," Feb. 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0
12. S. Ramírez, "FastAPI," software documentation. [Online]. Available: https://fastapi.tiangolo.com (accessed Nov. 2026).
13. Python Software Foundation, "heapq — Heap queue algorithm," *The Python Standard Library*, Python 3 docs. [Online]. Available: https://docs.python.org/3/library/heapq.html (accessed Nov. 2026).
14. PostGIS Project, "PostGIS manual: ST_DWithin." [Online]. Available: https://postgis.net/docs/ST_DWithin.html (accessed Nov. 2026).
15. Celery Project, "Celery: Distributed task queue," documentation. [Online]. Available: https://docs.celeryq.dev (accessed Nov. 2026). (Proposed worker queue; not yet deployed.)
16. National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015. (Pre-2021 standard.)
17. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015. (Pre-2021 standard.)

MetroFlow: Flood-Aware Rescue Routing

PBL Milestone 1 · ALGOCOM / SOFTENG · Team F4 · III-CCSAD · Nov 4, 2026

Project	MetroFlow
Team	F4 · Section III-CCSAD
Members & roles	Lapating, Khent U. (Lead Algorithm Architect) · De Vera, Josiah Ray S. (Backend / Systems) · Vergara, John Gabriel D. (Data / QA) · Felipe, Marck Aaron V. (Systems Analyst / Tech Lead)
Model	Waterfall
Repository	https://github.com/siahray/MetroFlow.git

The problem

- Typhoon flooding makes roads impassable **for some vehicles, not others**.
- A 28 cm unit and a 70 cm truck face different road networks.
- Dispatchers need: the **fastest route a given vehicle can actually drive**, from live water depths, or a clear “no safe route”.
- Depth-based road disruption and flood-aware emergency access are documented [6], [7].

Waterfall lifecycle

Phase	Artifact	This milestone
1. Requirements	BRD, FSD	Task 1: formal problem
2. Design	TSD (ADR-004 A*)	Tasks 2–4: algorithm, complexity, architecture, API
3. Implementation	`routing.py`, `hydrology.py`	DEV-ALGO-101 work package
4. Verification	Test cases, CI gates	Benchmark and edge-case assertions
5. Acceptance	UAT, sign-off	After Milestone 1

Each phase passes its exit criteria before the next begins; changes go through change requests.

Mathematical formulation

- **Input:** $G = (V, E)$; per edge ℓ_e (m), s_e (km/h), d_e (cm); query (s, t, c)
- **Constraint:** \(d_e
- **Objective:** minimise travel time
- **Output:** $(\text{status}, P^*, T^*)$

$$v_e = \max\left(\frac{s_e/3.6}{1+(d_e/10)^{1.8}},\ 0.5\right)$$

$$\min_{P:\ s\to t \text{ in } G_c} \sum_{e\in P} \frac{\ell_e}{v_e}$$

Depths from SCS-CN runoff [8].

Algorithm: clearance-constrained A*

Field	Detail
Paradigm	Graph theory: best-first search, min-heap [1], [2]
Baseline	Exhaustive path enumeration $O(n!)$; Dijkstra ($h \equiv 0$) [4]
Recurrence	$g(v) = \min_{(u,v)\in E_c}(g(u) + w_{uv})$, $f = g + h$
Heuristic	$h(v) = \mathrm{hav}(v, t)/v_{\max}$: **admissible and consistent** ⇒ exact, each node settled once [3]

Pseudocode (core loop)

```
g[s]=0; PQ={(h(s),0,s)}; settled={}
while PQ not empty:
    (f, gu, u) = pop_min(PQ)
    if u in settled or gu > g[u]: continue
    settled.add(u)
    if u == t: return path, g[t]
    for (v, len, speed, depth) in G[u]:
        if depth >= c: continue                  # clearance cut-off
        pen   = 1 + (depth/10)^1.8
        v_eff = max((speed/3.6)/pen, 0.5)
        g2    = gu + len / v_eff
        if g2 < g[v]: g[v]=g2; parent[v]=u; push(PQ,(g2+h(v), g2, v))
return NO_SAFE_ROUTE
```

Complexity

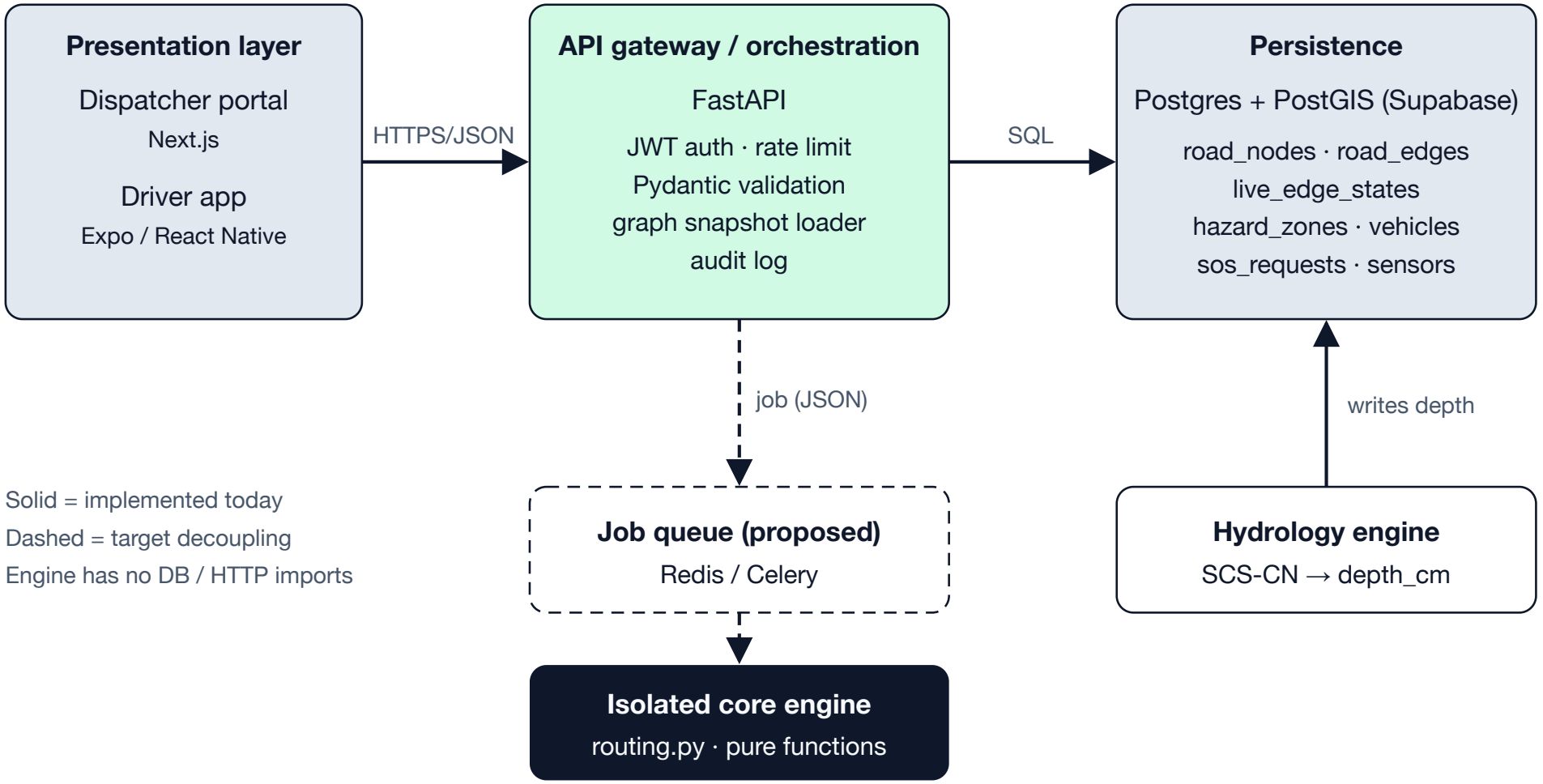
Measure	Derived	Measured (5,041 nodes)
Worst-case time	$O(m \log n)$: $\leq m + 1$ heap ops [1]	median 3.4 ms, p95 10.4 ms
Best / average	$\Omega(k \log n)$; sparse graph settles a region	33–43% fewer nodes than Dijkstra
Growth	near-linear on sparse grids	3.5×, 4.3× per 4× nodes
Space	$O(n + m)$	—
Optimality	exact (admissible, consistent)	0 mismatches in 120 queries

Honest finding: A* settles fewer nodes but is 18–52% slower than Dijkstra in wall time (costly haversine in Python). Master theorem n/a.

Scalability (3 V's)

V	Parameter	Mitigation
Volume	Pilot district graph $\rightarrow \sim 10^5$ nodes planned	Cache topology, load by bounding box (PostGIS) [14], CSR if needed
Velocity	Typhoon bursts; K searches per SOS	Worker pool + job queue; immutable depth snapshot
Variety	GeoJSON, sensors, reports, hazards	Normalise to `cm`, `km/h`, `m` at gateway; Pydantic validation

Decoupled architecture



Data contract: `POST /api/v1/algorithm/solve`

```
// request
{ "request_id": "uuid",
  "algorithm": "td_a_star",
  "start_node": 101, "target_node": 482,
  "vehicle": { "vehicle_id": 7,
               "clearance_cm": 28.0 },
  "options": { "ignore_hazards": false,
               "time_limit_ms": 250 } }
```

```
// response certificate
{ "status": "OK",   // NO_SAFE_ROUTE | TIMEOUT
  "result": { "path": [101,134,219,482],
              "travel_time_s": 412.6,
              "distance_m": 3180.4 },
  "metrics": { "nodes_settled": 348,
               "runtime_ms": 14.2,
               "optimal": true } }
```

OpenAPI 3.1 [11]; errors as RFC 9457 problem+json [9]; HTTP semantics per RFC 9110 [10].

Work packages & verification plan

- **Story 1** (FSD-FR-089): dispatcher runs per-vehicle searches to assign the lowest safe travel time.
- **Story 2** (FSD-FR-057): driver is rerouted on latest depths, or told there is no safe route.
- **DEV-ALGO-101** (`backend/routing.py`): edge cases – unknown node, `s=t`, disconnected, depth = clearance, bad speed.
- **Verification gate** (harness written and run; CI wiring pending): A* cost = Dijkstra on every query; p95 < 50 ms at N = 5,041 (measured 10.4 ms); settled ≤ Dijkstra.
- Scope fixed at the Design gate; changes via change request, not re-prioritisation.

Algorithm inventory

In-process algorithm	Derived	Measured
Flood-aware A* (primary)	$O(m \log n)$	3.4 ms @ 5k nodes
SCS-CN + ponding [8]	$O(1)$/node	60 ms @ 100k
Passability + hazard block	$O(1)$	0.23 µs
Urgency score + sort	$O(k \log k)$	101 ms @ 100k
SHA-256 audit chain [16]	$O(L)$	467 ms @ 100k
Turn-by-turn	$O(k)$	6.1 ms @ 10k
Sensor/report mismatch	$O(g)$	411 µs @ 1k gauges
Duplicate-neighbour map	$O(k^2)$	34 ms @ 200; 8.7 s @ 3.2k
Nearest-node snap	$O(n)$	50 ms @ 100k
Flood-state resolver (CR-004)	$O(\bar{o})$/edge	280 ms @ 100k edges
Evidence: digest, EXIF, polygon	$O(b)$; EXIF $O(e)$	2.2 ms/5 MB; 12 µs
Dispatch preview, K=10	$K \times$ A*	39 ms @ 5k nodes

Database-bound (local PostGIS)	Derived	Measured
Rate-limit upsert	$O(1)$	0.96 ms/call
SSE replay	$O(\log N + r)$	1.1 ms, flat to 500k rows
`ST_Intersects` hazards	$O(h \log m)$	2.4–3.3 ms, flat
Advisory upsert	$O(r)$	11 ms / 96 edges
`ST_DWithin` radius	claimed $O(\log m)$	**118 ms @ 50k: seq scan,** $O(m)$
JWT, SSE frame, seed build	$O(1)$–$O(n)$	22 µs; 2 µs; 0.17 ms

Findings: the impact-radius query ignores the spatial index; the duplicate map is quadratic (safe only at the 200-row page cap); A* settles 33–43% fewer nodes but runs 18–52% slower in Python. CR-004's resolver is implemented; routing doesn't read it yet. All 19 algorithms: Tasks 1–6. Dev-machine timings.

Defense highlights

- **Why decouple?** CPU-bound search starves request threads; pure function is fuzz-testable; immutable snapshot keeps optimality valid.
- **Exact vs heuristic:** our A* is exact (admissible + consistent); weighted A* only as a time-limited fallback.
- **Theory vs benchmark:** Big-O at Design predicts growth; CI at Verification measures constants and the average case.
- **Sparse data, not cold start:** default to modelled depth [8]; mark stale edges low-confidence.
- **Edge cases:** defined statuses, never exceptions; cycles harmless via `settled`.

References

1. Cormen *et al.*, *Introduction to Algorithms*, 4th ed., MIT Press, 2022.
2. Hart, Nilsson, Raphael, IEEE Trans. SSC, 4(2), 1968.
3. Dechter, Pearl, J. ACM, 32(3), 1985.
4. Dijkstra, Numer. Math., 1, 1959.
5. Bast *et al.*, Route planning in transportation networks, LNCS 9220, 2016.
6. Tsang, Scott, J. Transport Geography, 86, 2020.
7. Pregnolato *et al.*, Transp. Res. D, 55, 2017.
8. USDA NRCS, NEH Part 630, Ch. 10, 2004.
9. RFC 9457, Problem Details for HTTP APIs, 2023.
10. RFC 9110, HTTP Semantics, 2022.
11. OpenAPI Specification v3.1.0, 2021.
12. FastAPI documentation.
13. Python `heapq` documentation.
14. PostGIS manual, ST_DWithin.
15. Celery documentation.
16. NIST FIPS 180-4, Secure Hash Standard, 2015.
17. RFC 7519, JSON Web Token, 2015.